

Hostel Management & Complaint System

A

Minor Project Report

Submitted in partial fulfillment of the requirement for the award of degree of



Bachelor of Technology

in

Computer Science & Engineering

Submitted to

**RAJIV GANDHI PROUDYOGIKI VISHWAVIDYALAYA,
BHOPAL (M.P.)**



Guided by:

Prof. Pooja Katariya

Submitted By:

Aditya Pratap (0832CS231012)

Anirudh Bagdi (0832CS231023)

Dhruv Patidar (0832CS231061)

Himanshu Agrawal (0832CS231079)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

CHAMELI DEVI GROUP OF INSTITUTIONS

INDORE (M.P.) 452020

2025-26

Hostel Management & Complaint System

A Minor Project Report submitted to

Rajiv Gandhi Proudhyogiki Vishwavidyalaya, Bhopal

Bachelor of Technology

in

Computer Science and Engineering

by

Aditya Pratap (0832CS231012)

Anirudh Bagdi (0832CS231023)

Dhruv Patidar (0832CS231061)

Himanshu Agrawal (0832CS231079)

Under the guidance of

Prof. Pooja Katariya

(Asst. Professor)



Session: 2025-2026

Department of Computer Science & Engineering

Chameli Devi Group of Institutions, Indore 452020

(Madhya Pradesh)

DECLARATION

We certify that the work contained in this report is original and has been done by us under the guidance of my supervisor(s).

- a. The work has not been submitted to any other Institute for any degree or diploma.
- b. We have followed the guidelines provided by the Institute in preparing the report.
- c. We have conformed to the norms and guidelines given in the Ethical Code of Conduct of the Institute.
- d. Whenever we have used materials (data, theoretical analysis, figures, and text) from other sources, we have given due credit to them by citing them in the text of the report and giving their details in the references.

Name and Signature of Project Team Members:

S. No.	Enrollment No.	Name of students	Signature of students
1.	0832CS231012	Aditya Pratap	
2.	0832CS231023	Anirudh Bagdi	
3.	0832CS231061	Dhruv Patidar	
4.	0832CS231079	Himanshu Agrawal	

CHAMELI DEVI GROUP OF INSTITUTIONS, INDORE



CERTIFICATE

Certified that the Minor project report entitled, “**HOSTEL MANAGEMENT & COMPLAINT SYSTEM**” is a bonafide work done under my guidance by **Aditya Pratap (0832SC231012)**, **Anirudh Bagdi (0832CS231023)**, **Dhruv Patidar (0832CS231061)**, **Himanshu Agrawal (0832CS231079)** in partial fulfillment of the requirements for the award of degree of Bachelor of Technology in Computer Science and Engineering.

Date:

(Prof. Pooja Katariya)

Guide

(Prof. Radheshyam Acholiya)

Head of the Department

(Dr. Manoj Agrawal)

Project Coordinator

(Dr. Manish Shrivastava)

Principal, CDGI

CHAMELI DEVI GROUP OF INSTITUTIONS, INDORE

ACKNOWLEDGEMENT

We have immense pleasure in expressing our sincerest and deepest sense of gratitude towards our guide **Prof. Pooja Katariya** for the assistance, valuable guidance and co-operation in carrying out this Project work. We express our sincere gratitude to **Dr. Manoj Agrawal** (Associate Professor, Dept. of CSE, Major Project Coordinator) for his valuable guidance and constant support. We are developing this project with the help of Faculty members of our institute and we are extremely grateful to all of them. We also take this opportunity to thank the Head of the Department **Prof. Radheshyam Acholiya**, and the Principal of Chameli Devi Group of Institutions **Dr. Manish Shrivastava**, for providing the required facilities for the project work. We are greatly thankful to our parents, friends and faculty members for their motivation, guidance and help whenever needed.

Name and signature of team Members:

1. Aditya Pratap
2. Anirudh Bagdi
3. Dhruv Patidar
4. Himanshu Agrawal

TABLE OF CONTENTS

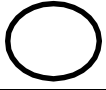
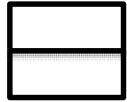
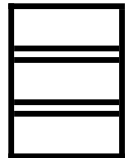
CONTENTS	Page No.
Title Page	
Declaration	I
Certificate by the Supervisor	II
Acknowledgement	III
Table of Contents	IV
List of Symbols	VIII
List of Figures	IX
List of Tables	X
Acronyms	XII
Abstract	XIII
Chapter 1: Introduction	1
1.1 Rationale	1
1.2 Project Overview	2
1.3 Objective	2
1.4 Scope	2
1.5 Roles & Responsibilities	3
1.6 Report Organization	3-4
Chapter 2: Study of Existing System and System Requirements	5
2.1 Identification of Need	5
2.1.1 Existing System & its drawbacks	5-8
2.2 Preliminary Investigation	9
2.2.1 Proposed System & its advantages	9
2.3 Feasibility Study	9
2.3.1 Technical Feasibility	9
2.3.2 Operational Feasibility	10
2.3.3 Economical Feasibility	10
2.4 Schedule & Milestone	11
2.4.1 Gantt Chart	11
2.4.2 Project Schedule Table	12
2.5 Resources and Budget	12
2.6 Risk Management	13

2.7 My Contribution	13-14
Chapter 3: Literature Review	15
3.1 Introduction	15
3.2 Related Work	15-18
3.2.1 Traditional Hostel Management Systems	15
3.2.2 ERP-Based Hostel Modules	16
3.2.3 Commercial Hostel and PG Management Applications	16
3.2.4 Web-Based Complaint Management Systems	17
3.2.5 Role-Based Management Systems	18
3.3 Summary of Related Work (in a table)	18
3.4 Research Gap	19
Chapter 4: System Analysis	20
4.1 Software Requirement Specification	20
4.1.1 External Interface Requirement	20
4.1.1.1 User Interface	21-23
4.1.1.2 Hardware Interface	23
4.1.1.3 Software Interface	24
4.1.1.4 Communication Interface	24
4.2 Functional Requirements	24-27
4.3 Non-Functional Requirements	27
4.3.1 Reliability	27
4.3.2 Availability	27
4.3.3 Security	27
4.3.4 Maintainability	27
4.3.5 Performance	28
4.4 My Contribution in System Analysis	28
Chapter 5: System Analysis Tools	29
5.1 Introduction	29
5.2 Design Overview	29
5.2.1 Data Flow Diagram	29-30
5.2.2 Use-case Diagram	31
5.2.3 Class Diagrams	32
5.3 Database Design	33
5.3.1 Entity Relationship Diagram	33

5.3.2 Conversion of E-R Diagram into Tables	34
5.3.3 Normalization Details	34-39
5.3.4 Table Structure	39-42
5.3.5 Data Dictionary	43-46
5.4 System Architectural Design (Block Diagram)	47-49
5.5 Detailed Description of Components (Modules)	50
5.5.1 Module 1	50
5.5.2 Module 2	50
5.5.3 Module 3	51
5.5.4 Module 4	51
5.5.5 Module 5	52
5.5.6 Module 6	52
5.5.7 Module 7	53
5.5.8 Module 8	53
5.5.9 Module 9	54
5.5.10 Module 10	54
5.5.11 Module 11	54
5.5.12 Module 12	55
5.5.13 Module 13	55
5.6 User Interface Design	56
5.6.1 Description of the User Interface	56
5.6.2 Screen Images	57-60
5.7 My Contribution	61
Chapter 6: Software Test Documentation (STD)	62
6.1 Introduction	62
6.1.1 System Overview	62
6.1.2 Test Approach	62
6.1.3 Testing Objectives	63
6.2 Test Plan	63
6.2.1 Features to be Tested	63
6.2.2 Features not to be Tested	63
6.3 Test Cases	64
6.3.1 Unit Testing	64
6.3.2 Functional Testing	65

6.3.3 System Testing	65-66
6.3.4 Integration Testing	66
6.3.5 Validation Testing	67
6.4 My Contribution	67-68
Chapter 7: Conclusion and Future Scope	69
References	71
Appendix List	72
Appendix A - Source Code	73
Appendix B - User Manual	74-76
Appendix C - Database Schema	77-82
Appendix D - Additional Results	83-85
Published Papers	-

LIST OF SYMBOLS

S. No.	Symbol	Name	Purpose
1.	Rectangle (ER diagram) 	Entity	Represents a database entity
2.	Circle 	Process	Represents a system process
3.	Arrow 	Data Flow	Shows data movement direction
4.	Open-ended Rectangle 	Data Store	Represents stored data
5.	Oval 	Use Case	Represents a system function
6.	Stick Figure 	Actor	Represents a system user
7.	Rectangle (System Boundary) 	System Boundary	Defines system scope
8.	Diamond 	Relationship	Shows relationship between entities
10.	Oval (ER Diagram) 	Attribute	Represents entity attribute
11.	Line 	Association	Shows connection or interaction
12.	Rectangle (ClassDiagram) 	Class	Represents a class
13.	Block 	Architecture Layer	Represents a system layer

LIST OF FIGURES

S. No.	Figure Name	Figure No.	Page No.
1.	Gantt chart	2.1	11
2.	Dataflow diagram (zero level)	5.1	29
3.	Dataflow diagram (first level)	5.2	30
4.	Dataflow diagram (second level)	5.3	30
5.	Use case diagram	5.4	31
6.	Class diagram	5.5	32
7.	ER diagram	5.6	33
8.	System Architectural Design	5.7	49
9.	Home Page	5.8	57
10.	Centralized Administrative Overview Dashboard	5.9	57
11.	User Profile and Security Management Interface	5.10	58
12.	Complaints Management and Maintenance Dispatch Hub	5.11	58
13.	Visitor Inquiries and Lead Conversion Console	5.12	59
14.	Role-Based User Management and Bulk Registration Sheet	5.13	59
15.	Public Hostel Inquiry and Accommodation Form	5.14	60
16.	Student Unified Dashboard and Information Portal	5.15	60

LIST OF TABLES

S. No.	Table Name	Table No.	Page No.
1.	Existing system & its drawback	2.1	6-8
2.	Project table	2.2	12
3.	Budget estimation	2.3	13
4.	Risk management	2.4	13
5.	Summary of Existing Systems and Their Limitation	3.1	18-19
6.	Student table	5.1	35
7.	Admin table	5.2	35
8.	Complaint table	5.3	36
9.	Warden table	5.4	36
10.	Chief Warden table	5.5	37
11.	Room table	5.6	37
12.	Staff table	5.7	37
13.	Mess table	5.8	38
14.	Department table	5.9	38
15.	Building table	5.10	39
16.	Table structure of student	5.11	39
17.	Table structure of room	5.12	40
18.	Table structure of Admin	5.13	40
19.	Table structure of mess	5.14	40
20.	Table structure of chief warden	5.15	40
21.	Table structure of Complaint	5.16	41
22.	Table structure of warden	5.17	41
23.	Table structure of staff	5.18	42
24.	Table structure of department	5.19	42
25.	Table structure of building	5.20	42
26.	Data dictionary of student	5.21	43
27.	Data dictionary of room	5.22	43
28.	Data dictionary of admin	5.23	44
29.	Data dictionary of mess	5.24	44
30.	Data dictionary of chief warden	5.25	44
31.	Data dictionary of complaint	5.26	45

S. No.	Table Name	Table No.	Page No.
32.	Data dictionary of warden	5.27	45
33.	Data dictionary of staff	5.28	46
34.	Data dictionary of department	5.29	46
35.	Data dictionary of building	5.30	46
36.	Unit Testing	6.1	64
37.	Functional Testing	6.2	65
38.	System Testing	6.3	65-66
39.	Integration Testing	6.4	66
40.	Validation Testing	6.5	67

Acrony

Acronym	Meaning
API	Application Programming Interface
B. Tech	Bachelor of Technology
JSON	Binary JSON
DBMS	Database Management System
DFD	Data Flow Diagram
ERD	Entity-Relationship Diagram
ERP	Enterprise Resource Planning
FAB	Floating Action Button
HTTP / HTTPS	Hypertext Transfer Protocol / Secure
JSON	JavaScript Object Notation
JWT	JSON Web Token
MERN	MongoDB, Express.js, React.js, Node.js
SPA	Single Page Application
SRS	Software Requirement Specification
STD	Software Test Documentation
TAT	Turnaround Time
UI / UX	User Interface / User Experience
URL	Uniform Resource Locator

ABSTRACT

The Hostel Management & Complaint System is designed to modernize and streamline hostel operations that are traditionally managed through manual and semi-digital processes. Conventional methods often result in inefficiencies such as delays in complaint resolution, poor record maintenance, lack of transparency, and communication gaps between students and administrators. Students frequently face difficulties in accessing notices, submitting requests, tracking approvals, and availing essential hostel services, while administrators struggle with handling paperwork, monitoring activities, and managing approvals in a timely manner.

To address these challenges, this project proposes a comprehensive web-based Hostel Management System that centralizes all hostel-related activities on a single digital platform. The system enables students to view notices, lodge and track complaints, apply for leave, register visitors, request services such as laundry and medical assistance, and raise emergency alerts when required. Administrators can efficiently manage rooms, staff, mess operations, fees, complaints, and approvals through a centralized dashboard, ensuring better coordination and faster decision-making.

The system significantly reduces paperwork, improves communication between stakeholders, and ensures transparency and accountability in hostel operations. By digitizing records and automating routine processes, the proposed solution enhances security, operational efficiency, and overall user experience. Its modular and scalable design allows for future enhancements such as mobile application support, real-time notifications, analytics, and reporting features, making it suitable for long-term institutional adoption

Chapter-1

Introduction

The Hostel Management & Complaint System is a web-based application designed to simplify and automate the process of registering and resolving hostel-related complaints. Traditionally, complaints are recorded manually, leading to delays, lack of tracking, and poor accountability. This system provides a centralized platform where students can submit complaints online, track their status, and receive updates.

The main objective of this project is to reduce complaint resolution time, improve transparency, and maintain proper records. The system supports multiple user roles such as students, wardens, maintenance staff, and administrators. It ensures better communication between students and hostel authorities and improves hostel management efficiency.

This chapter introduces The Hostel Management & Complaint System and explains the background, purpose, scope, and structure of the project. It highlights the need for an automated complaint management solution in hostels and provides an overview of how the proposed system addresses existing problems. The chapter also defines the objectives of the project, identifies user roles, and explains how the report is organized.

1.1 Rationale

In most educational institutions, hostels accommodate a large number of students who depend on hostel management for essential facilities such as water supply, electricity, cleanliness, furniture, internet connectivity, and security. Any disruption in these facilities directly affects the daily life and academic performance of students.

Traditionally, hostel complaints are managed manually using complaint registers or verbal communication. These methods are inefficient because complaints may be forgotten, records can be lost, and there is no proper tracking mechanism. Students are often unaware of whether their complaint has been noted or resolved. Due to the absence of transparency and accountability, complaint resolution becomes slow and unreliable.

1.2 Project Overview

The Hostel Management & Complaint System is a web-based software application developed to manage and monitor hostel - related complaints efficiently. The system allows students to register complaints online by providing relevant details such as complaint category, description, and supporting images.

Once a complaint is registered, it is forwarded to the hostel authorities. Wardens can review complaints and assign them to the appropriate maintenance staff. The maintenance staff can update the complaint status after resolving the issue. The administrator oversees the entire system, manages users, and generates reports.

The system uses a centralized database to store all complaint-related data securely. This ensures easy access to records, quick retrieval of information, and long-term data preservation.

1.3 Objective

The primary objectives of the Hostel Management & Complaint System are:

- a) To reduce delays in complaint resolution.
- b) To maintain an accurate and permanent record of complaints.
- c) To ensure transparency in the complaint handling process.
- d) To improve coordination between students, wardens, and maintenance Staff.

1.4 Scope

The scope of this project includes the complete digital workflow of hostel gate pass management:

- a. The scope of this project is limited to the management of hostel-related complaints within an educational institution.
- b. The system supports activities such as complaint registration, complaint assignment, status tracking, and resolution.
- c. The project does not include financial transactions, biometric systems, or integration with external campus management systems. However, the system is scalable and can be enhanced in the future to include mobile applications, notification services, and advanced analytics.

1.5 Roles And Responsibility

The Hostel Management & Complaint System defines the following user roles:

Student: Registers complaints, views complaint status, and receives update.

Warden: Reviews complaints, assigns them to maintenance staff, and monitors

Staff: Resolves assigned complaints and updates status.

Admin: Manages users, complaint categories, and generates reports.

Each role has specific access rights to ensure security and proper system usage.

1.6 Report Organization

This report is organized into seven chapters, each focusing on a specific aspect of the Hostel Management & Complaint System to provide a systematic understanding of the project.

Chapter 1: Introduction

This chapter presents the project overview, problem background, objectives, scope, and structure of the report.

Chapter 2: Study of Existing System and System Requirements

This chapter analyzes the existing hostel management system, identifies its limitations, and defines the system requirements and feasibility.

Chapter 3: Project Planning

This chapter explains project planning activities including scheduling, milestones, resource allocation, cost estimation, and risk management.

Chapter 4: System Analysis

This chapter describes the system architecture, database design, and detailed design of all functional modules.

Chapter 5: System Analysis Tools

This chapter discusses the modeling tools used such as Data Flow Diagrams, Use Case Diagram, Class Diagram, ER Diagram, and module descriptions.

Chapter 6: Software Test Documentation (STD)

This chapter outlines the testing strategy, test plan, objectives, and various testing levels to ensure system reliability.

Chapter 7: Conclusion and Future Scope

This chapter summarizes the project work and highlights future enhancements and scope of improvement.

Chapter-2

Study of Existing System and System Requirements

This chapter presents a detailed study of the existing Hostel Management & Complaint System and identifies the need for a more efficient digital solution. It discusses the common problems faced by hostel students, such as issues related to water supply, electricity, cleanliness, internet connectivity, furniture repair, and security, which often remain unresolved due to the absence of a proper complaint handling mechanism. The chapter also examines the limitations and drawbacks of the existing manual and semi-digital systems, including delayed responses, lack of transparency, poor tracking, and limited customization in currently available applications. Based on this analysis, the need for a computerized complaint management system is established to improve communication, accountability, complaint tracking, and overall hostel administration efficiency.

2.1 Identification of Need

In hostels, students frequently face issues related to basic facilities such as water supply, electricity, cleanliness, furniture repair, internet connectivity, and security. These problems require immediate attention to ensure a comfortable living environment. However, the absence of a proper complaint management mechanism results in delayed resolutions and frustration among students.

The need for a computerized complaint management system arises to ensure that complaints are properly recorded, tracked, and resolved within a reasonable time. A digital system helps in maintaining transparency, accountability, and efficiency in hostel administration.

2.1.1 Existing System & its drawbacks

The existing system of Hostel Management & Complaint System is mostly manual. Students report their complaints verbally to the hostel warden or write them in a complaint register maintained in the hostel office. In some cases, complaints are communicated through phone calls or informal messages.

Drawbacks of Existing System

1. NAV Hostel^[1]

- Limited Customization,
- integration challenges with existing systems.

Study of Existing System and System Requirements

2. eZee FrontDesk^[2]

- Limited Customization for Students complaints
- More focused on hostels / PGsthan Education hostels.

3. Accsoft- Hostel Module^[3]

- The hostel module seems basic; doesn't clearly list complaint / grievance – tracking or maintenance request workflows.

S. No.	Existing App Name	Features	Limitations
1.	NAV Hostel [1]	• Automates room allocation, attendance • Fee management maintain requests. • Provide mobile access for students and staff.	• Limited Customization , integration challenges with existing systems.
2.	eZee FrontDesk [2]	• Cloud - based hostel & Property management. • Room/bed allocation & reservations. • House Keeping, invoicing, reports • Multi-device access (web + Mobile)	• Limited Customization for Students complaints. • More focused on hostels / PGsthan Education hostels
3.	Accsoft Hostel Module [3]	• Web-based requests & approvals (PHP/MySQL) • Role-based access & real-time tracking. • Reporting & notification	• The hostel module seems basic; doesn't clearlylist complaint/ grievance–tracking or maintain request workflows.

Study of Existing System and System Requirements

4.	MasterSoft ERP Hostel [4]	• Hostel room allocation management. • Hostel fee and attendance management.	• Complex interface for beginners. • Complaint management is limited.
5.	Hostel Software [5]	• Room allotment and vacancy tracking. • Fee collection and reporting.	• Limited grievance management. • Dependent on internet connectivity.
6.	DHMS (Digital Hostel Management System) [6]	• AI chatbot assistance. • Digital complaint registration and tracking.	• Research-based prototype. • Complex implementation process.
7.	E-Based Hostel Management System [7]	• Online student registration. • Digital hostel record management.	• Basic complaint handling features. • Limited scalability for large hostels.
8.	Freshworks Complaint Management [8]	• Complaint ticket generation. • Automated notifications and tracking.	• Not hostel-specific. • Requires customization for educational institutions.

Study of Existing System and System Requirements

9.	Zoho Desk Complaint Management [9]	• Ticket-based complaint handling. • Workflow automation and status tracking.	• Designed for customer support systems. • Advanced features require paid plans.
10.	Complaint Management System [10]	• Online complaint submission. • Complaint status tracking.	• Limited hostel-related features. • No room allocation integration.

Table 2.1 Existing System & drawbacks

2.2 Preliminary Investigation

A preliminary investigation was conducted by interacting with hostel students, wardens, and maintenance staff. The investigation revealed that students often hesitate to repeatedly follow up on complaints, while hostel authorities find it difficult to manage and prioritize multiple complaints simultaneously.

The investigation clearly indicated the requirement for a centralized, automated system that could streamline the complaint handling process and improve coordination among all stakeholders.

The following activities were performed during the preliminary investigation:

- Interaction with hostel students to understand common problems such as water leakage, power failure, internet issues, cleanliness, and furniture damage.
- Discussion with hostel wardens to study how complaints are currently received, recorded, and forwarded to maintenance staff.
- Observation of the manual complaint register to analysis how complaints are written, tracked, and closed.
- Identification of delays in complaint resolution due to lack of follow-up and unclear responsibility.
- Analysis of communication gaps between students, wardens, and maintenance staff.

Study of Existing System and System Requirements

2.2.1 Proposed System & its advantages

The proposed Hostel Management & Complaint System is a web-based application that allows students to register complaints online and track their status in real time. Complaints are stored in a centralized database and can be accessed by authorized users only.

Advantages of the Proposed System

The proposed system offers the following advantages:

- Easy and quick complaint registration
- Real-time tracking of complaint status
- Reduced Paperwork and manual efforts
- Secure storage of complaint records
- Faster resolution of complaints
- Improved communication between students and hostel staff

2.3 Feasibility Study

A feasibility study is an important step in system development which determines whether the proposed system is practical and achievable. The feasibility study of the Hostel Management & Complaint System is analysis under three main categories: technical feasibility, operational feasibility, and economic feasibility.

The purpose of this study is to ensure that the system can be developed using available resources, can be operated smoothly by users, and can be implemented at a reasonable cost.

2.3.1 Technical Feasibility

Technical feasibility analysis whether the proposed system can be developed using the available technology, hardware, and software resources.

The Hostel Management & Complaint System is technically feasible due to the following reasons:

- The system is developed using well-known and widely used technologies such as HTML, CSS, JavaScript, React for frontend development.
- Backend development is done using Python-based frameworks like Django or Flask, which are reliable and scalable.
- MySQL is used as the database, which is open-source, secure, and capable of handling large data efficiently.

Study of Existing System and System Requirements

The system can run on standard computers and does not require high-end hardware.

- Internet connectivity is easily available in hostels and institutions.
- Developers have sufficient knowledge and technical skills to implement the system. Hence, from a technical point of view, the proposed system is completely feasible.

2.3.2 Operational Feasibility

Operational feasibility examines whether the proposed system can be successfully used and accepted by the users in real working conditions.

The proposed system is operationally feasible because:

- The system provides a simple, user-friendly interface that can be easily used by students and hostel staff.
- Minimum training is required for users to operate the system.
- The system reduces manual work and simplifies the complaint handling process.
- Students can easily track the status of their complaints, increasing transparency.
- Hostel authorities can manage complaints more efficiently through a centralized platform.
- The system improves coordination and communication among students, wardens, and maintenance staff.

Therefore, the system is expected to be readily accepted by users and smoothly integrated into hostel operations.

2.3.3 Economical Feasibility

Economic feasibility analysis whether the cost of developing and maintaining the system is justified by its benefits.

The Hostel Management & Complaint System is economically feasible due to the following reasons:

- The system uses open-source software, which reduces development costs.
- No additional expensive hardware is required.
- Development is carried out using existing infrastructure such as computers and internet.
- Maintenance cost is minimal.
- The system reduces paperwork and administrative effort, resulting in long-term cost savings.

Thus, the overall cost of the project is low, making it suitable for educational institutions.

Study of Existing System and System Requirements

2.4 Schedule & Milestone

Project planning is an essential phase in the software development life cycle. It defines how the project will be executed, controlled, and completed within a specific time and resource constraint. Proper planning helps in identifying tasks, allocating resources, managing risks, and ensuring timely completion of the project.

This chapter explains the planning strategy adopted for the Hostel Management & Complaint System, including project scheduling, milestones, required resources, budget estimation, and risk management.

2.4.1 Gantt Chart

The Gantt chart represents the overall project schedule and illustrates the timeline of activities along with their dependencies. Each task was planned sequentially, starting from requirement analysis to final documentation. The chart helped in monitoring progress, identifying task overlaps, and ensuring timely completion of each phase.

Project Planning – Gantt chart

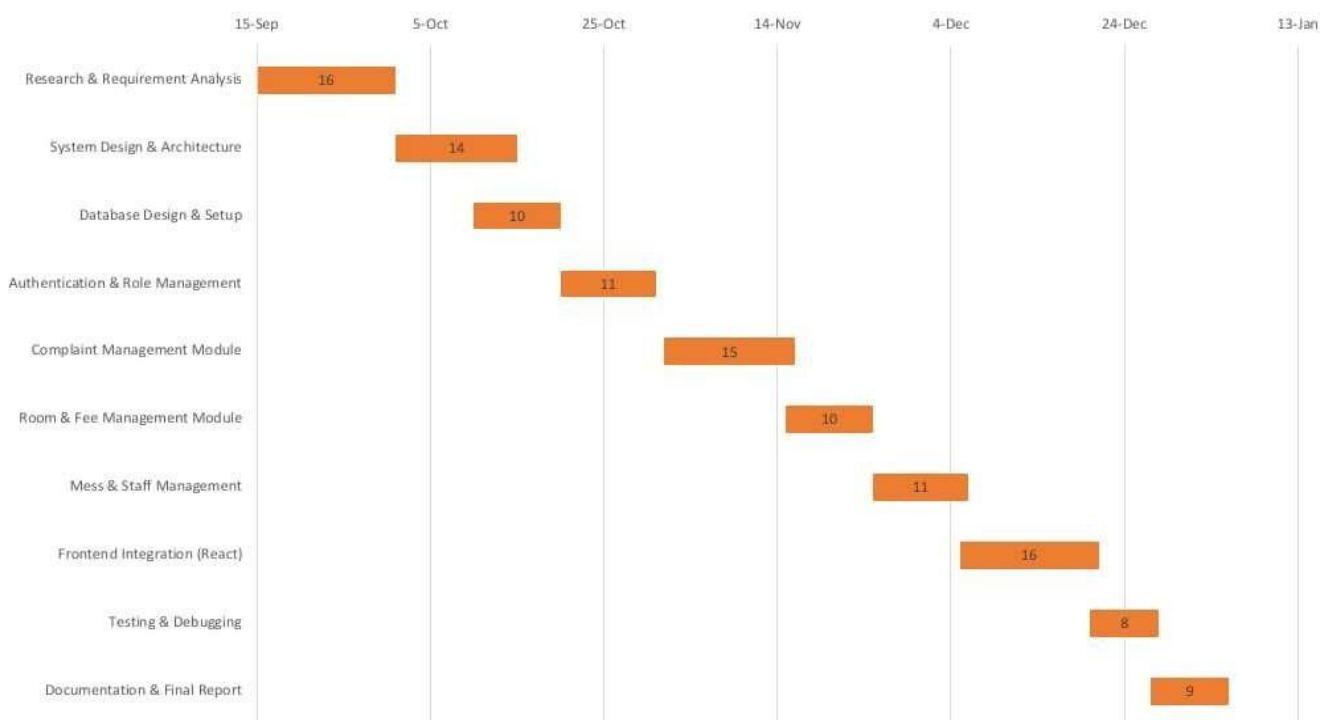


Fig 2.1: Gantt chart

Study of Existing System and System Requirements

2.4.2 Project Schedule Table

Phase	Task Description	Duration
1	Research & Requirement analysis	2 Week
2	System design & architecture	2 Week
3	Database design & setup	1 Week
4	Authentication & role management	1 Week
5	Complaint management module	1 Week
6	Room & fee management module	1 Week
7	Mess And Staff Management	1 Week
8	Frontend Integration	2 Week
9	Testing & Debugging	1 Week
10	Documentation & Final Reeport	1 Week

Table 2.2: Project table

This schedule ensured balanced workload distribution and allowed sufficient time for testing and refinement.

2.5 Resources and Budget

The project was developed using open-source tools and technologies, which minimized overall cost while maintaining scalability and performance.

Resources Used

a) Hardware:

- Laptop/Desktop, Smartphone (for Complaint Registration)

b) Software:

- Frontend: React.js
- Backend: Node.js, Express.js
- Database: MongoDB
- Tools: VS Code, GitHub, Postman

c) Human Resources:

- 4 Student Developers

Study of Existing System and System Requirements

Budget Estimation

S. No.	Resource	Cost
1	Software Tools	Free (Open Source)
2	Development Environment	Existing Systems
3	Hosting	Minimal / Free Tier
4	Total Estimated Cost	Very Low

Table 2.3: Budget estimation

The project is economically viable as it does not require licensed software or specialized hardware.

2.6 Risk Management

Risk management was considered to identify potential challenges during development and deployment. Preventive measures were planned to minimize their impact.

Risk	Impact	Mitigation Strategy
Requirement changes	Medium	Modular design and flexible database schema
Technical complexity	Medium	Use of well- documented frameworks
Data loss	High	Regular backups and logging
Security vulnerabilities	High	JWT authentication and role-based access
Time constraints	Medium	Strict milestone tracking using Gantt chart

Table 2.4: Risk management

2.7 My Contribution

The successful completion of Chapter 2 involved equal contribution from all four team members. Each member was assigned specific responsibilities during the study of the existing system, requirement analysis, and feasibility study.

• Aditya Pratap:

Studied the existing hostel complaint handling system and identified its major drawbacks. Collected information about current manual processes and documented problems faced by students and hostel authorities.

Study of Existing System and System Requirements

- **Anirudh Bagdi:**

Conducted preliminary investigation by interacting with hostel students and wardens. Analyzed user requirements and expectations from the proposed system and helped in identifying the need for automation.

- **Dhruv Patidar:**

Prepared the details of the proposed system and listed its advantages over the existing system. Also contributed to defining system objectives and scope based on gathered requirements.

- **Himanshu Agrawal**

Performed feasibility analysis of the project including technical, operational, and economic feasibility. Assisted in documentation work and formatting of Chapter 2 content.

Chapter-3

Literature Review

3.1 Introduction

Literature review is an important part of software project development because it helps in understanding existing systems, technologies, and research related to the proposed project. It provides information about the advantages, limitations, and working methods of current hostel management and complaint handling systems.

In many educational institutions, hostel activities such as complaint registration, room management, and communication between students and wardens are still managed manually or through partially digital systems. These methods often lead to delayed complaint resolution, poor record maintenance, lack of transparency, and communication problems.

Various hostel management and complaint management systems have been developed to improve these processes. However, many existing systems mainly focus on administrative tasks and provide limited support for real-time complaint tracking, role-based management, and automated workflows.

This chapter reviews the existing systems, technologies, and research works related to hostel management and complaint handling. The study helps in identifying the limitations of current systems and establishes the need for a centralized web-based Hostel Management & Complaint System.

3.2 Related Work

3.2.1 Traditional Hostel Management Systems

Traditional hostel management systems are mostly manual and depend on physical registers, written applications, and verbal communication for managing complaints and hostel activities. In many educational institutions, students report issues directly to wardens or hostel staff, and records are maintained manually. Although these systems are simple and low-cost, they create several problems such as delayed complaint resolution, poor record maintenance, lack of transparency, and difficulty in tracking complaint status. Manual systems also increase paperwork and reduce communication efficiency between students, wardens, and maintenance staff. Due to the absence of digital tracking and centralized data management, handling a large number of complaints becomes difficult and time-consuming.

3.2.2 ERP-Based Hostel Modules

Many educational institutions use Enterprise Resource Planning (ERP) systems to manage academic and administrative activities, including hostel operations. These ERP-based hostel modules provide features such as room allocation, fee management, attendance tracking, and student record maintenance through a centralized platform. Although these systems help in organizing hostel data efficiently, their complaint management functionality is usually limited to basic complaint registration and record keeping. Most ERP modules do not provide real-time complaint tracking, automated notifications, role-based escalation, or direct communication between students, wardens, and maintenance staff. In addition, ERP systems are often costly, complex to maintain, and difficult to customize according to the specific requirements of hostel administration.

3.2.3 Commercial Hostel and PG Management Applications

Several commercial hostel and Paying Guest (PG) management applications are available in the market. Applications such as NAV Hostel, eZee FrontDesk, and Accsoft Hostel Module provide digital hostel administration features.

NAV Hostel: NAV Hostel offers facilities such as room allocation, attendance management, fee tracking, and mobile accessibility. It helps institutions maintain hostel records digitally. However, the system provides limited customization and faces integration issues with existing institutional systems. [1]

eZee FrontDesk: eZee FrontDesk is a cloud-based hostel and property management solution. It supports room reservations, housekeeping management, billing, and reporting. The system works efficiently for hotels and commercial accommodations but provides limited support for educational hostel complaint management and institutional workflows. [2]

Accsoft Hostel Module: Accsoft Hostel Module is a web-based application developed using PHP and MySQL technologies. It supports role-based access, request management, and approval systems. However, the complaint management functionality appears basic and lacks advanced grievance tracking, escalation handling, and maintenance workflow automation. [3]

MasterSoft ERP Hostel: MasterSoft ERP Hostel provides room allocation, attendance monitoring, fee management, and occupancy tracking for educational institutions. However, its interface can be complex for new users, and complaint management features are limited. [4]

Hostel Software: Hostel Software supports room allotment, vacancy tracking, fee collection, and student record management. However, it offers limited grievance handling capabilities and depends heavily on internet connectivity. [5]

DHMS (Digital Hostel Management System): DHMS integrates AI chatbot support with digital complaint registration and tracking. However, it is primarily a research prototype and requires complex implementation for practical deployment. [6]

E-Based Hostel Management System: This system enables online student registration, room allocation, and digital hostel record management. However, it provides only basic complaint handling features and faces scalability limitations. [7]

Freshworks Complaint Management: Freshworks offers complaint ticket generation, status tracking, and automated notifications. However, it is not specifically designed for hostel environments and requires customization for institutional use. [8]

Zoho Desk Complaint Management: Zoho Desk provides ticket-based complaint handling, workflow automation, and status tracking. However, it is primarily designed for customer support systems, and advanced features require paid plans. [9]

Complaint Management System: Complaint Management System allows users to submit complaints online and track their resolution status. However, it lacks hostel-specific features such as room allocation and hostel administration integration. [10]

3.2.4 Web-Based Complaint Management Systems

Web-based complaint management systems are becoming increasingly popular because they allow users to submit complaints online and track their progress digitally. These systems reduce paperwork and improve transparency in complaint handling.

Most web-based complaint systems include features such as:

- Online complaint registration
- Complaint status tracking
- User authentication
- Database management
- Administrative dashboards

However, many existing systems are developed for general-purpose complaint handling and are not specifically designed for hostel environments. They often lack hostel-specific features such as room-based complaint categorization, mess-related complaints.

3.2.5 Role-Based Management Systems

Modern management applications use role-based access control to improve security and workflow management. In such systems, different users are assigned specific permissions according to their responsibilities.

For example:

- Students can register and track complaints.
- Wardens can review and assign complaints.
- Staff members can update work status.
- Administrators can monitor overall system activities.

Role-based systems improve accountability, security, and operational efficiency. The proposed Hostel Management & Complaint System also follows a role-based architecture to ensure secure and organized complaint handling.

3.3 Summary of Related Work

Existing System	Main Features	Limitations
NAV Hostel [1]	Room allocation, attendance management, fee tracking, mobile support	Limited customization and integration support
eZee FrontDesk [2]	Cloud-based hostel management, room booking, housekeeping, reporting	More suitable for hotels and PGs than educational hostels
Accsoft Hostel Module [3]	Role-based access, web-based management, approval workflow	Limited complaint tracking and maintenance workflow
MasterSoft ERP Hostel [4]	Room allocation, fee management, attendance management	Complex interface, limited complaint management
Hostel Software [5]	Room allotment, vacancy tracking, fee collection, reporting	Limited grievance management, internet dependency
DHMS [6]	AI chatbot, digital complaint registration and tracking	Prototype stage, complex implementation

E-Based Hostel Management System [7]	Online registration, digital record management	Basic complaint handling, limited scalability
Freshworks Complaint Management [8]	Ticket generation, automated notifications, tracking	Not hostel-specific, customization required
Zoho Desk Complaint Management [9]	Ticket handling, workflow automation, status tracking	Customer-support focused, paid advanced features
Complaint Management System [10]	Complaint submission, status tracking	Limited hostel features, no room allocation integration

Table 3.1: Summary of Existing Systems and Their Limitation

3.4 Research Gap

The literature review shows that existing hostel management systems mainly focus on room allocation, attendance, fee collection, and administrative tasks. Complaint management is often treated as a secondary feature and lacks proper workflow automation.

The following research gaps were identified in the existing systems:

- Absence of real-time complaint tracking mechanisms.
- Lack of communication between students, wardens, and maintenance staff.
- No proper complaint escalation system for unresolved issues.
- Limited hostel-specific complaint categorization.
- Insufficient transparency and accountability in complaint handling.
- Lack of centralized digital records for hostel complaints.
- Poor integration of role-based workflows and notification systems.
- Existing systems are either too basic or too commercially focused for educational institutions.

To overcome these limitations, the proposed Hostel Management & Complaint System introduces a centralized web-based platform with role-based access, complaint tracking, status updates, notification services, and escalation management. The system improves operational efficiency, reduces manual work, and enhances communication among all stakeholders involved in hostel administration.

Chapter-4

System Analysis

System analysis is an important phase in software development that focuses on understanding user requirements, identifying system functionalities, and defining how the proposed system will operate. It helps in studying the problems of the existing system and designing an improved solution that fulfills user needs efficiently. The Hostel Management & Complaint System is designed to automate hostel-related activities such as complaint registration, complaint tracking, room management, mess management, and communication between students and hostel authorities. The system analysis phase identifies both functional and non-functional requirements of the proposed system and defines how different users like students, wardens, staff, chief wardens, and administrators interact with the application. This chapter explains the Software Requirement Specification (SRS), external interfaces, system requirements, and overall functional behavior of the system. It also describes security, performance, reliability, and communication requirements necessary for smooth and efficient operation of the Hostel Management & Complaint System.

4.1 Software Requirement Specification

The Software Requirement Specification outlines the functional, non-functional, and interface requirements that define the expected behavior and constraints of the Hostel Management & Complaint System. The SRS ensures a common understanding of the system for developers, evaluators, and stakeholders by detailing how the system should operate, what features it must support, and how users will interact with it.

4.1.1 External Interface Requirement

External interface requirements describe how the Hostel Management & Complaint System interacts with users, software platforms, and communication services. This section explains the interfaces through which different stakeholders such as students, wardens, staff, chief wardens, and administrators access the system. It also defines how the system communicates with external services like web browsers, databases, and notification services. Clearly defining these interfaces ensures smooth interaction, usability, compatibility, secure access, and reliable data exchange between the system and its external environment.

4.1.1.1 User Interface

The system will offer a responsive, intuitive, and role-based interface accessible through modern web browsers. Each user type will have a customized dashboard that aligns with their workflow and responsibilities.

• Student Interface

The Student Interface is designed to be simple and user-friendly so that students can easily interact with the system without any technical difficulty. After successful login, the student is redirected to the student dashboard.

1. Student login using unique credentials
2. Dashboard displaying total complaints, pending complaints, and resolved complaints
3. Option to register a new complaint
4. Selection of complaint type such as Room, Mess, Electricity, Water, Cleanliness, etc.
5. Enter complaint description and upload supporting images if required
6. View status of submitted complaints (Pending, In-Progress, Resolved)
7. Receive notifications related to complaint updates.
8. Track pending and resolved complaints

• Complaint Interface

The Complaint Interface handles all complaint-related activities in the system. It acts as a central module where complaints are created, viewed, updated, and closed.

1. Complaint registration form with fields like category, description, priority, and attachment.
2. Auto-generation of complaint ID.
3. Status update feature.
4. Complaint history tracking.
5. Search and filter complaints based on status, date, and category.
6. Complaint registration form with fields like category, description, priority, and attachment.
7. Auto-generation of complaint ID.
8. Status update feature.
9. Complaint history tracking.

• Mess Interface

The Mess Interface is used for handling complaints related to food quality, hygiene, menu issues, and staff behavior Ability to approve or reject forwarded applications.

1. Separate category for mess-related complaints
2. Daily or weekly complaint tracking
3. Option for mess in-charge to view and respond to complaints
4. Status update after issue resolution.

• Room Interface

The Room Interface manages complaints related to hostel rooms such as furniture damage, lighting issues, fans, switches, and cleanliness.

1. Room-specific complaint registration
2. Automatic mapping of complaint with room number
3. Assignment to maintenance staff
4. Status tracking and resolution confirmation
5. Super Admin Interface.

• Warden Interface

The Warden Interface is used by hostel wardens to manage complaints related to their assigned hostel.

1. View complaints hostel-wise.
2. Assign complaints to maintenance or mess staff.
3. Update complaint priority.

• Chief warden interface

The Chief Warden Interface provides a higher-level view of all hostel complaints And overall system performance.

1. View all complaints across hostels.
2. Monitor complaint resolution time.
3. Generate reports and statistics.
4. Escalate unresolved complain.

• Staff Interface

The Staff Interface is designed for maintenance and support staff responsible for resolving complaints.

1. View assigned complaints
2. Update work progress
3. Change complaint status after resolution
4. Add remarks or resolution details

• Admin Interface

The Admin Interface provides complete control over the system and is accessible only to authorized administrators

1. Manage users and roles
2. Add or remove complaint categories
3. Monitor overall system activity
4. Generate detailed reports
5. Ensure system security and data integrity.

4.1.1.2 Hardware Interface

The Hostel Management & Complaint System is a **web-based application** and does not require any specialized hardware. It works using commonly available computing devices and standard peripherals. The system interacts with hardware only through basic input and output devices.

Client-side:

- a) Computer / Laptop / Mobile Device used to access the system through a web browser.
- b) Internet connectivity required to connect to the web-based application

Server-side:

1. Server Machine stores application logic and database handles user requests and complaint processing
2. The system does not directly control any hardware devices platform independent, No dependency on special hardware All hardware interaction is indirect, through:

- Web browser
- Operating system

4.1.1.3 Software Interface

The system requires smooth integration between backend services, the database, and third-party libraries.

- (a) Operating System: platform-independent (Windows, Linux, macOS).
- (b) Backend: Node.js with Express.js framework.
- (c) Frontend: React.js for building dynamic user interfaces.
- (d) Database: MongoDB (document-based, scalable, lightweight).
- (e) Authentication: JSON Web Tokens (JWT).
- (f) Browser Compatibility: Chrome, Firefox, Edge, Safari.

4.1.1.4 Communication Interface

- (a) Client-Server Communication: RESTful API over HTTP/HTTPS.
- (b) Data Formats: JSON for request/response.
- (c) Authentication Header: Bearer tokens for secure access.
- (d) Notification Channels: Email/SMS APIs (optional).
- (e) Network Requirements: The application supports both LAN (Local Area Network) and WAN (Wide Area Network) environments.
- (f) Security Protocols:
- (g) HTTPS for encrypted transport
- (h) Secure login and authentication mechanisms are used.

4.2 Functional Requirements

These requirements describe what the system must do.

Student Functional Requirements

- (a) Register and login securely into the system
- (b) View personal profile details
- (c) Register a new complaint

- (d) Select complaint category (mess, room, electrical, water, cleanliness, etc.)
- (e) Provide complaint description and upload supporting details (if any)
- (f) View the status of submitted complaints
- (g) Track complaint resolution progress
- (h) Receive updates on complaint status
- (i) Logout securely from the system.

Warden Functional Requirements

- (a) Login securely into the system
- (b) View complaints related to assigned hostel
- (c) Verify and review student complaints
- (d) Assign complaints to maintenance staff
- (e) Set complaint priority based on urgency
- (f) Monitor complaint status and resolution time
- (g) Add remarks or instructions for staff
- (h) Update complaint status
- (i) Communicate with students regarding complaints
- (j) Logout securely from the system

Chief warden Functional Requirements

- (a) Login securely into the system
- (b) View all hostel complaints
- (c) Escalate unresolved or delayed complaints
- (d) Provide feedback or directions to wardens
- (e) Logout securely from the system

Staff Functional Requirements

- a) Login securely into the system
- b) View assigned complaints
- c) View complaint details and location
- d) Update work progress
- e) Mark complaint as resolved after completion

- f) Add resolution remarks
- g) Report issues if complaint cannot be resolved

Room module Functional Requirements

- (a) Maintain room details such as room number, block, floor, and capacity
- (b) Associate students with their respective rooms
- (c) Allow wardens/admin to update room information
- (d) Allow room-wise complaint categorization
- (e) Support room availability and allocation records
- (f) Allow admin to add, edit, or delete room records.

Mess module Functional Requirements

- (a) Maintain mess details such as mess name and menu
- (b) Allow students to register mess-related complaints
- (c) Categorize complaints related to food quality, hygiene, timing, and staff behavior.
- (d) Allow wardens to review mess complaints
- (e) Assign mess complaints to responsible staff
- (f) Track resolution status of mess complaints

Complaint module Functional Requirements

- (a) Generate a unique complaint ID for each complaint
- (b) Store complaint details such as category, description, date, and status
- (c) Allow complaints to be updated and tracked
- (d) Support complaint prioritization

System-Level Requirements

- (a) Login securely into the system
- (b) Manage user accounts (add, update, delete)
- (c) Assign roles (student, warden, staff, admin)
- (d) Manage complaint categories
- (e) Manage hostel, room, and mess details
- (f) Monitor system activity

- (g) Backup and maintain database
- (h) Ensure system security.

4.3 Non-Functional Requirements

These define how well the system performs.

4.3.1 Reliability

- (a) The system should be available during normal hostel working hours.
- (b) Complaint data should remain consistent and accurate.
- (c) Database must maintain integrity even during server restarts or failures

4.3.2 Availability

- (a) The system shall be available to users 24×7 through internet connectivity.
- (b) System downtime shall be minimal.
- (c) Users shall be able to access the system at any time except during scheduled maintenance.

4.3.3 Security

- (a) The system shall provide secure login and authentication for all users.
- (b) Role-based access control shall be enforced.
- (c) JWT tokens ensure session security.
- (d) Sensitive data shall be protected from unauthorized access.
- (e) User sessions shall be securely managed.
- (f) HTTPS ensures protection against eavesdropping.

4.3.4 Maintainability

- a) The system shall be easy to maintain and update.
- b) Well-documented APIs support easy debugging.
- c) MongoDB schemas are flexible for future enhancements.
- d) System maintenance shall require minimal effort.
- e) Code comments, version control (Git), and structured directories increase maintainability.

4.3.5 Performance

- a) The system shall respond to user requests within acceptable time limits.
- b) Multiple users shall be able to access the system simultaneously.
- c) System should scale to handle hundreds of requests concurrently.
- d) MongoDB indexing ensures fast read/write operations.
- e) Complaint submission and retrieval shall be fast and efficient.

4.4 Contribution in System Analysis

During the system analysis phase of the Hostel Management & Complaint System, We actively participated in understanding the existing system, identifying problems, and defining system requirements. Our contributions listed below:

Aditya Pratap:

Worked on security of data & reliability of system

- (a) Ensured reliable complaint storage and retrieval without data loss.
- (b) Contributed to defining secure authentication.

Anirudh Bagdi:

Contributed to defining the core system requirements and overall structure of the System Analysis chapter.

- (a) Identified and documented functional requirements for all system roles.
- (b) Analyzed security and performance requirements of the proposed system.
- (c) Ensured alignment between system workflow and documented requirement

Dhruv Patidar:

Focused on user interaction and interface-related aspects of system analysis.

- (a) Documented user interface and hardware interface requirements.
- (b) Described role-based dashboards and usability considerations.
- (c) Assisted in refining operational feasibility aspects.

Himanshu Agrawal:

Worked on technical interfacing and quality-related system requirements.

- (a) Documented software and communication interface requirements.
- (b) Contributed to non-functional requirements such as maintainability and portability.
- (c) Reviewed performance and availability constraints of the system

Chapter-5

System Analysis Tool

5.1 Introduction

This chapter presents the system analysis tools used to model and represent the functional and structural aspects of the Hostel Management & Complaint System. These tools provide a clear understanding of system behavior, data flow, user interaction, and internal structure. Diagrams such as Data Flow Diagrams (DFD), Use Case Diagram, and Class Diagram are used to visualize the system at different abstraction levels.

5.2 Design Overview

The system design focuses on modularity, role-based access, and clear separation of responsibilities among users. Different analysis tools are used to represent the system from multiple perspectives—data movement, user interactions, and class structure. The following sections explain these tools in detail.

5.2.1 Data Flow Diagram

a) DFD Level Zero

The Zero-Level Data Flow Diagram represents the entire system as a single process and shows interactions with external entities such as Student, admin, Room, Complaint, Mess, staff, warden, department, building and Chief warden as a **USER**. It provides a high-level overview of how data flows into and out of the system without showing internal processing.



Fig 5.1: Dataflow diagram (zero level)

b) First Level DFD

The First-Level Data Flow Diagram decomposes the main system into major functional processes such as Application & complaint, Room & Mess, Complaint Management, Hostel Database. It explains how data flows between these processes and external entities in greater detail.

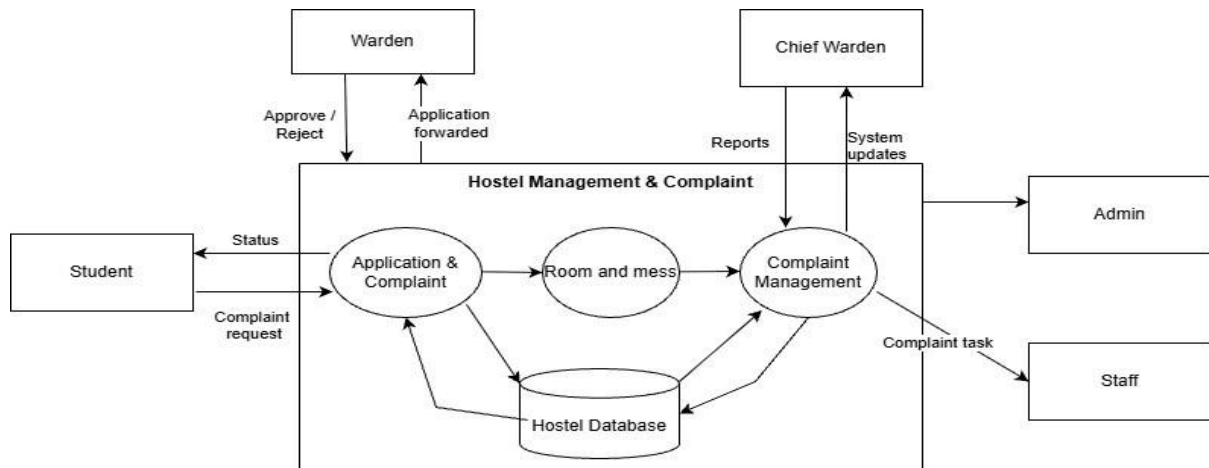


Fig 5.2: Dataflow diagram (first level)

c) Second Level DFD

The Second-Level Data Flow Diagram provides a detailed breakdown of internal operations, including Review Decide Complaints, approval workflow, QR verification, entry/exit logging, and report generation. It also shows how data is stored and retrieved from system logs and databases.

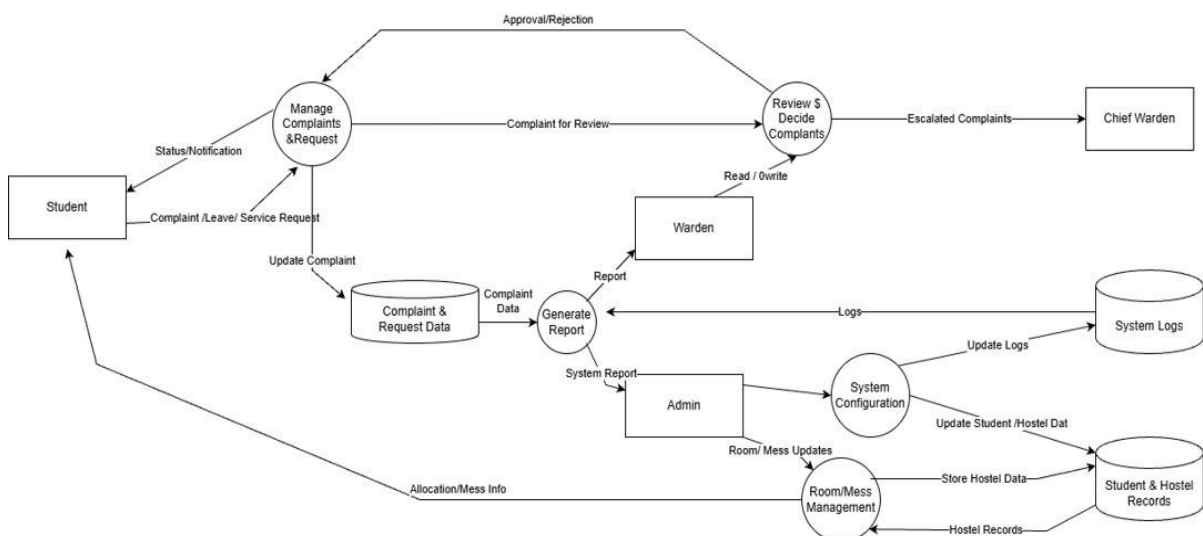


Fig 5.3: Dataflow diagram (second level)

5.2.2 Use-Case Diagram

The Use Case Diagram represents the functional requirements of the Hostel Management & Complaint System from the user's perspective. It illustrates the interactions between various actors and the system, clearly defining system boundaries and role-based responsibilities. The primary actors involved are Student, Admin, Warden, Staff, and chief warden. Students can login, submit Complaints, view notices & Mess menu, apply leaves, Track complaint status. wardens are responsible for handle Complaints, approve / reject leave, or Response to emergencies and assign a complaint to staff. Also, they approve / reject visitor's entry and serious category complaints like health issues they escalated complaints to chief warden.

Chief-warden are responsible to review Escalated Complaints, monitor hostel operations, and taking Critical Decisions. This diagram helps in understanding how different users interact with the system functionalities.

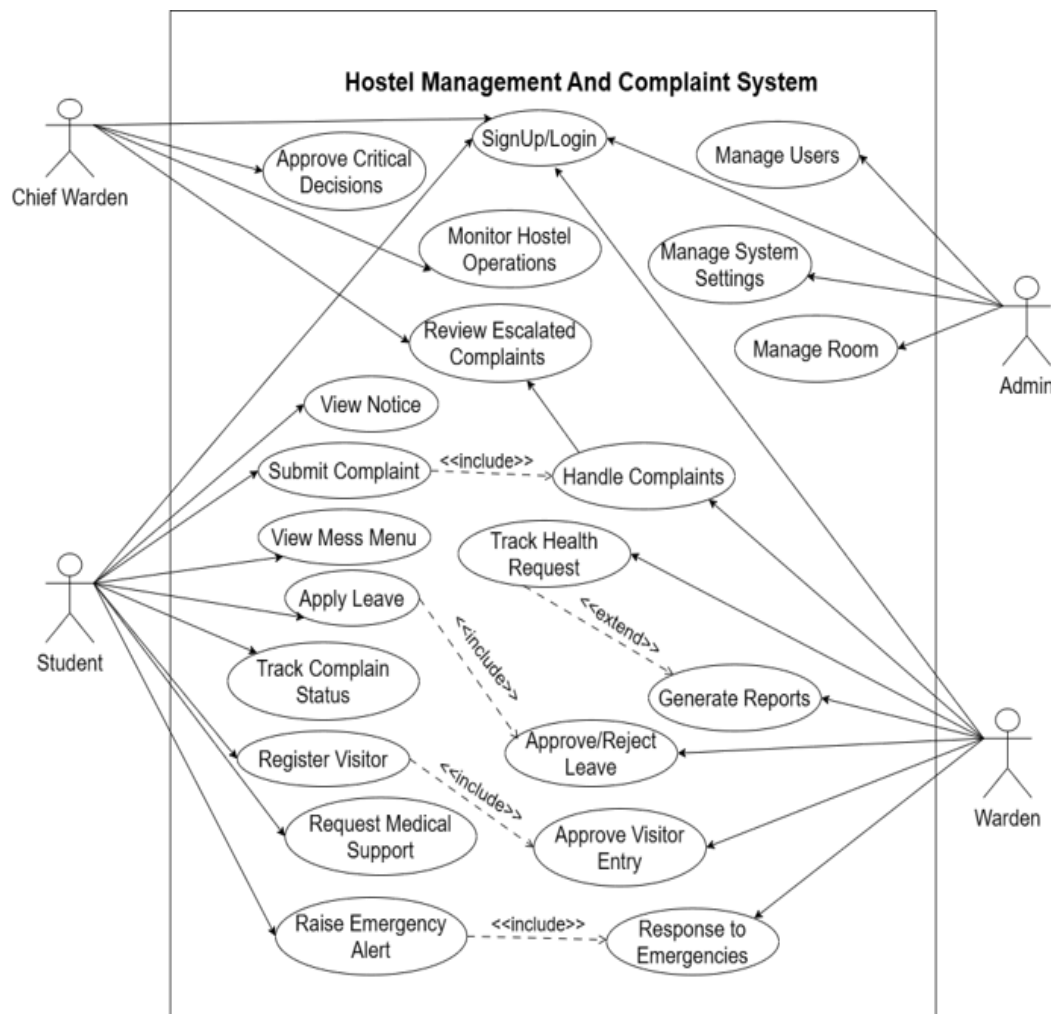


Fig 5.4: Use case Diagram

5.2.3 Class Diagram

The Class Diagram represents the static structure of the Hostel Management & Complaint System. It shows the classes involved in the system, their attributes, methods, and the relationships between them. Major classes include student, complaint, staff, warden, chief warden, admin, leaveRequest, visitorRequest, healthRequest, EmergencyAlert, notification.

The diagram highlights how users are linked to hostels. Associations between classes define data flow and responsibility sharing, while methods represent system operations such as create complaints, resolve complaints, update Status, and approval visitors. This diagram serves as the foundation for database design and system implementation.

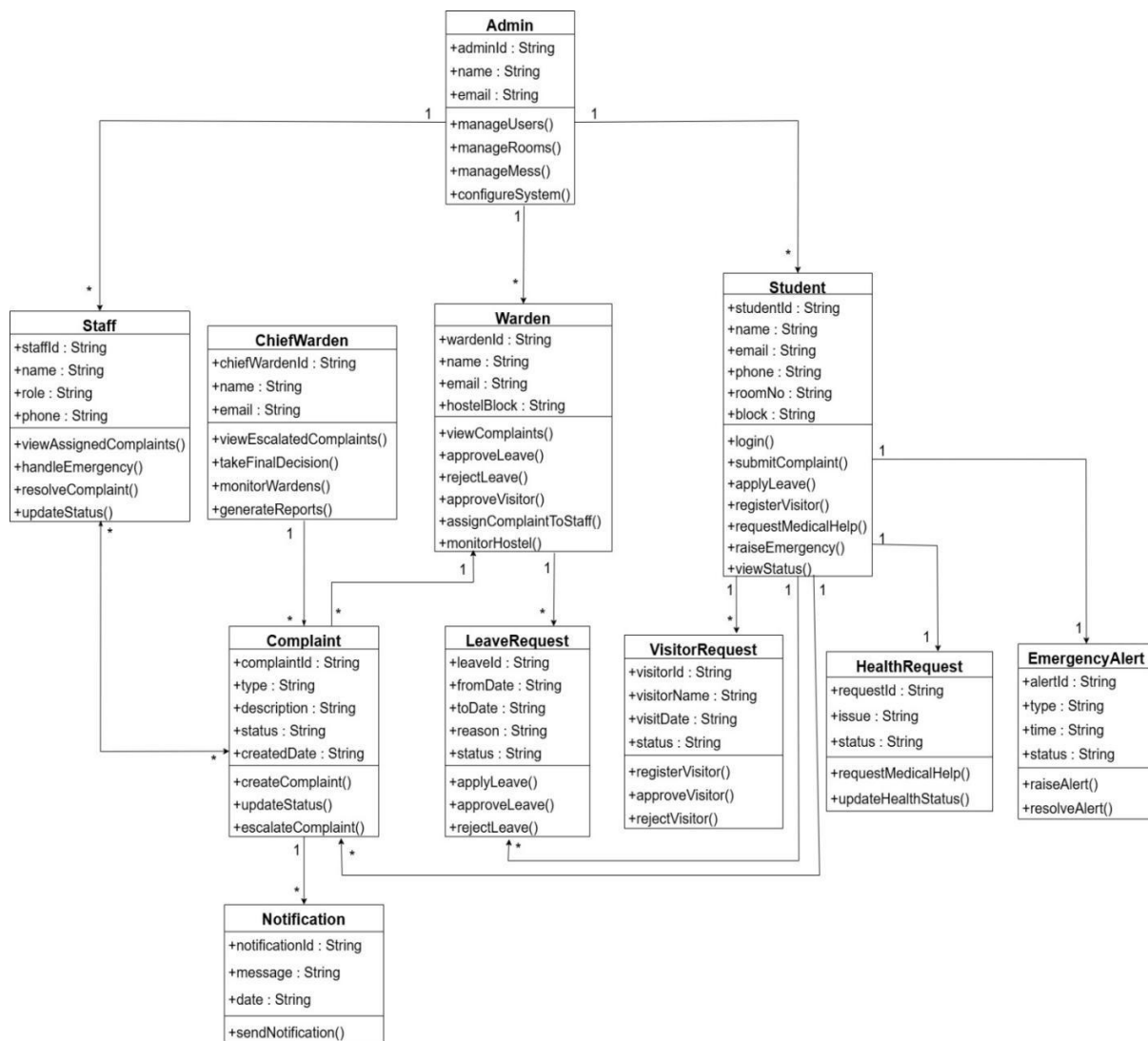


Fig 5.5: Class diagram

5.3 Database Design

This section describes the database design of the Hostel Management & Complaint System. The database is designed to store and manage information related to users, complaints, room, approvals, and movement logs in a structured and normalized manner. The design process includes creation of an Entity Relationship (E-R) diagram, conversion of entities into relational tables, normalization up to Third Normal Form (3NF), detailed table structures, and a comprehensive data dictionary.

5.3.1 Entity Relationship Diagram

The Entity Relationship (E-R) Diagram represents the conceptual design of the database used in the Hostel Management & Complaint System. It identifies the major entities involved in the system such as Student, complaint, room, Mess, Warden, Chief warden, staff, department, building and admin along with their attributes. The diagram also defines the relationships between these entities, such as submit, manages, Assigns and supervise. This diagram serves as the foundation for converting the conceptual model into a relational database schema.

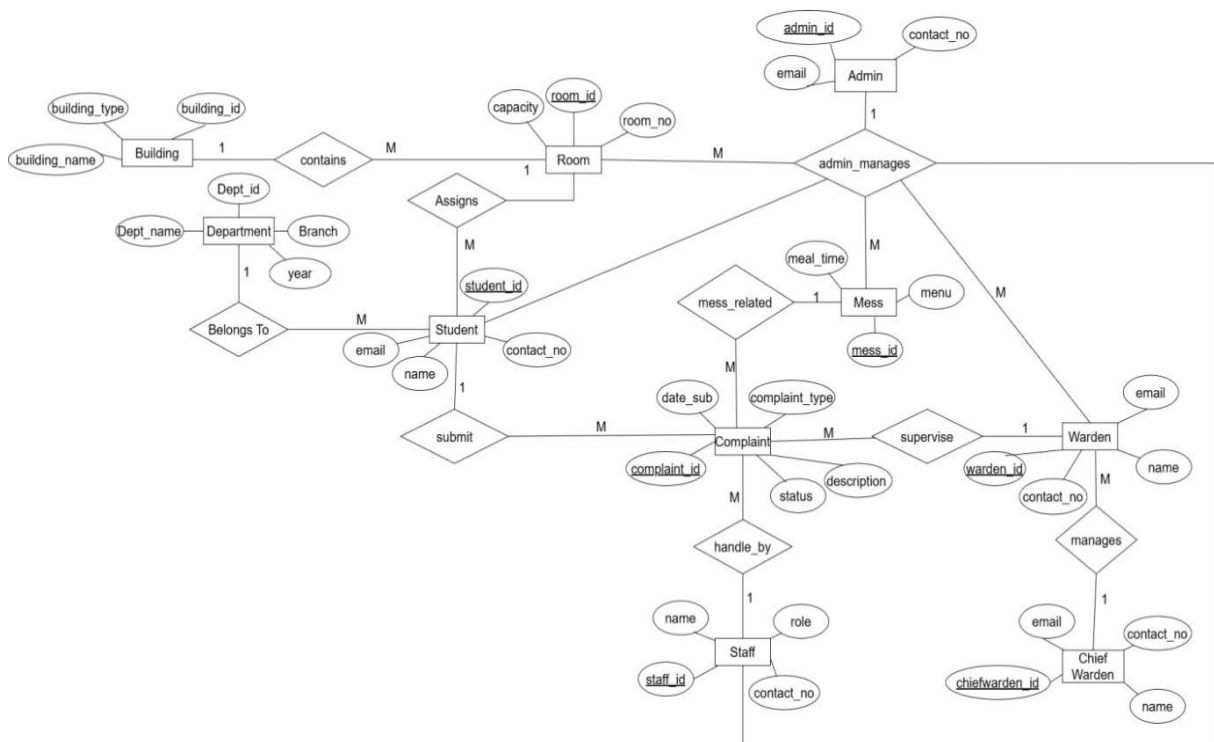


Fig 5.6: ER diagram

5.3.2 Conversion of E-R Diagram into Tables

The E-R diagram was converted into a relational schema by mapping each entity to a corresponding table. Each table is assigned a unique primary key to uniquely identify records. Entity attributes were converted into table columns with appropriate data types. Relationships between entities were implemented using foreign keys to ensure referential integrity. One-to-many relationships were handled by placing the foreign key of the parent entity in the child table. This conversion ensures structured data storage, consistency, and efficient querying of the database.

5.3.3 Normalization Details

Normalization was performed to eliminate redundancy and ensure data consistency in the database. All tables were normalized up to Third Normal Form (3NF).

For each table, normalization was applied as follows:

- First Normal Form (1NF):

All attributes are atomic and contain no repeating groups or multi-valued fields.

- Second Normal Form (2NF):

Each non-key attribute is fully functionally dependent on the entire primary key.

- Third Normal Form (3NF):

No transitive dependencies exist. Related data is stored in separate tables and linked using foreign key.

Each table in the database (Student, Complaint, Room, Mess, Warden, Chief Warden staff and Admin) follows these normalization rules to ensure data integrity and efficient database operations.

a) **Student Table:**

Field Name	Data Type	Key
Student_id	INT	PK
Name	VARCHAR(50)	—
Email	VARCHAR(50)	UNIQUE
Contact_no	VARCHAR(11)	—
Year	Year	—
Room_no	INT	FK

Table 5.1 Student table**Normalization:**

1NF: Atomic attributes (student_id, name, year, room_no, room_id, email, contact_no,)

2NF: PK = student_id; all fields depend entirely on the PK.

3NF: No transitive dependencies. All non-primary keys depend only one PK.

b) **Admin Table:**

Field Name	DATA TYPE	Key
Admin_id	INT	PK
Name	VARCHAR(50)	—
Email	VARCHAR(50)	NOT NULL
Contact_no	VARCHAR(12)	—

Table 5.2 Admin table**Normalization:**

1NF: Atomic attributes (admin_id, name, email, contact_no)

2NF: PK = room_id only one PK contains Room table.

3NF: No transitive dependencies. All non-primary keys depend only one PK.

c) Complaint Table:

Field Name	DATA TYPE	Key
Complaint_ID	INT	PK
Student_ID	INT	FK
Complaint_TYPE	VARCHAR(50)	—
Description	VARCHAR(50)	—
Status	VARCHAR(50)	—
Complaint_DATE	DATE	—
Mess_ID	INT	FK
Warden_ID	INT	FK
Staff_ID	INT	FK

Table 5.3 Complaint table**Normalization:**

1NF: Atomic attributes (complaint_id, student_id, complaint_type, description, status, complaint_date, mess_id, warden_id, staff_id)

2NF: PK = complaint_id; only one PK contains complaint table.

3NF: No transitive dependencies. All non-primary keys depend only one PK

d) Warden table:

Field Name	Data Type	Key
Warden_id	INT	PK
Email	VARCHAR(100)	—
Contact_info	VARCHAR(15)	—
Building_assign	VARCHAR(50)	FK
Chiefwarden_id	INT	FK

Table 5.4 Warden table**Normalization:**

1NF: Atomic attributes (warden_id, chiefwarden_id, email, contact_no, block_assign)

2NF: PK = warden_id; only one PK contains warden table.

3NF: No transitive dependencies. All non-primary keys depend only one PK.

e) Chief warden

Field Name	Data Type	Key
Chiefwarden_id	INT	PK
Email	VARCHAR(50)	—
Contact_no	VARCHAR(11)	—

Table 5.5 Chief warden

Normalization:

1NF: Atomic attributes (chiefwarden_id, email, contact_no)

2NF: PK = chiefwarden_id; only one PK contains chief-warden table.

3NF: No transitive dependencies. All non-primary keys depend only one PK.

f) Room Table

Field Name	Data Type	Key
Room_id	INT	PK
Room_no	INT	NOT NULL
Capacity	INT	—

Table 5.6 Room table

Normalization:

1NF: Atomic attributes (room_id, room_no, block , capacity);

2NF: PK = room_id; only one PK contains Room table.

3NF: No transitive dependencies. All non-primary keys depend only one PK

g) Staff Table

Field Name	Data Type	Key
Staff_id	INT	PK
Name	VARCHAR(50)	—
Role	VARCHAR(50)	—
Assigned_block	VARCHAR(50)	FK
Contact_no	VARCHAR(50)	—

Table 5.7 Staff table

Normalization:

1NF: Atomic attributes (staff_id, name, contact_no, role, assigned_block); 2NF: PK = staff_id; only one PK contains staff table.

3NF: No transitive dependencies. All non-primary keys depend only one PK.

h) Mess Table

Field Name	DATA TYPE	Key
Mess_ID	INT	PK
Menu	VARCHAR(40)	—
Meal_Time	DATETIME	—

Table 5.8 Mess table

Normalization:

1NF: Atomic attributes (mess_id, menu, meal_time);

2NF: PK = mess_id; only one PK contains Mess table.

3NF: No transitive dependencies. All non-primary keys depend only one PK.

i) Department

Field	Type	Key
Department_ID	INT	PK
Department_name	VARCHAR(40)	NOT NULL
Branch	VARCHAR(40)	—

Table 5.9 Department table

Normalization:

1NF: Atomic attributes (Department_id, Department_name, Branch);

2NF: PK = Department_id; only one PK contains Department table.

3NF: No transitive dependencies. All non-primary keys depend only one PK.

j) **Building**

Field	Type	Key
Building_ID	INT	PK
Building_name	VARCHAR(40)	NOT NULL
Building_type	VARCHAR(40)	—

Table 5.10 Building table**Normalization:**

1NF: Atomic attributes (Building_id, Building_name, Building_type);

2NF: PK = Building_id; only one PK contains Building table.

3NF: No transitive dependencies. All non-primary keys depend only one PK.

5.3.4 Table Structure

This section describes the structure of the database tables used in the system. Each table is defined with its fields, data types, primary keys, and foreign keys. Proper constraints such as primary key, foreign key, and uniqueness are applied to maintain data integrity and enforce relationships among entities. The table structures ensure efficient storage and retrieval of data required for gate pass requests, approvals, verification, and reporting.

- Student:

Field Name	Data Type	Key	Description
Student_id	INT	PK	Unique identifier for student
Name	VARCHAR(50)	—	Student name
Email	VARCHAR(50)	UNIQUE	Student email address
Contact_no	VARCHAR(11)	—	Student contact number
Year	Year	—	Student Course year
Room_no	INT	NOT NULL	Student room no.

5.11 Table structure of student

- Room:

Field Name	Data Type	Key	Description
Room_id	INT	PK	Unique room identity
Room_no	INT	—	Students room no.
Capacity	INT	—	Room capacity of student

5.12 Table structure of room

- Admin:

Field Name	Data Type	Key	Description
Admin_id	INT	PK	Unique identifier for admin
Name	VARCHAR(50)	—	Admin name
Email	VARCHAR(50)	NOT NULL	Admin email
Contact_no	VARCHAR(12)	—	Admin contact number

5.13 Table structure of Admin

- Mess:

Field Name	Data Type	Key	Description
Mess_ID	INT	PK	ID of Mess
Menu	VARCHAR(40)	—	Menu of Mess
Meal_Time	DATETIME	—	Meal Timing

5.14 Table structure of mess

- Chief warden:

Field Name	Data Type	Key	Description
Chiefwarden_id	INT	PK	Unique identity of chief - warden
Email	Varchar2(50)	—	Email of chief warden
Contact_no	Varchar2(11)	—	Contact no. of Chief warden

5.15 Table structure of chief warden

- Complaint:

Field Name	Data Type	Key	Description
Complaint_ID	INT	PK	ID of complaint
Student_ID	INT	FK	ID of student
Complaint_TYPE	VARCHAR(50)	—	Type of complaint
Description	VARCHAR(50)	—	Description of problem
Status	VARCHAR(50)	—	Status of problem
Complaint_DATE	DATE	—	Date of complaint
Mess_ID	INT	FK	ID of mess
Warden_ID	INT	FK	ID of warden
Staff_ID	INT	FK	ID of staff

5.16 Table structure of complaint

- Warden:

Field Name	Data Type	Key	Description
Warden_id	INT	PK	Unique identity of warden
Email	VARCHAR(100)	—	Warden email
Contact_info	VARCHAR(15)	—	Contact number of warden
Building_assign	VARCHAR(50)	—	Assigned building of warden
Chiefwarden_id	INT	FK	Unique identity of chief- warden

5.17 Table structure of warden

- Staff:

Field Name	Data Type	Key	Description
Staff_id	INT	PK	Unique staff identity
Name	VARCHAR(50)	—	Name of staff members
Role	VARCHAR(50)	—	Role of staff
Assigned block	VARCHAR(50)	FK	Assigned block to staff
Contact_no	VARCHAR(50)	FK	Contact info of staff

5.18 Table structure of staff

- Department

Field Name	Data Type	Key	Description
Department_ID	INT	PK	Unique department identity
Department_name	VARCHAR(40)	—	Name of department
Branch	VARCHAR(40)	—	Name of branch

5.19 Table structure of department

- Building

Field Name	Data Type	Key	Description
Building_ID	INT	PK	Unique building identity
Building_name	VARCHAR(40)	—	Name of building
Building_type	VARCHAR(40)	—	Types of Building (boys/girls)

5.20 Table structure of building

5.3.5 Data Dictionary

The data dictionary provides detailed information about each attribute used in the database. It includes the field name, data type, constraints, and a brief description explaining the purpose of the attribute. The data dictionary serves as a reference for understanding the database schema and helps developers and administrators maintain consistency during development and future enhancements.

a) Student:

Field Name	Data Type	Constraints	Description
Student_id	INT	PK	Unique identifier for student
Name	VARCHAR2(50)	NOT NULL	Student name
Email	VARCHAR2(50)	UNIQUE	Student email address
Contact_no	VARCHAR2(11)	—	Student contact number
Year	Year	NULL NOT	Student Course year
Room_no	INT	NOT NULL	Student room no.

5.21 Data dictionary of student

b) Room:

Field Name	Data Type	Constraints	Description
Room_id	INT	PK	Unique room identity
Room_no	INT	—	Students room no.
Capacity	INT	—	Room capacity of student

5.22 Data dictionary of Room

c) Admin:

Field Name	Data Type	Constraints	Description
Admin_id	INT	PK	Unique identifier for admin
Name	VARCHAR(50)	—	Admin name
Email	VARCHAR(50)	NOT NULL	Admin email
Contact_no	VARCHAR(12)	—	Admin contact number

5.23 Data dictionary of Admin

d) Mess:

Field Name	Data Type	Constraints	Description
Mess_ID	INT	PK	ID of Mess
Menu	VARCHAR(40)	—	Menu of Mess
Meal_Time	DATETIME	—	Meal Timing

5.24 Data dictionary of Mess

e) Chief Warden:

Field Name	Data Type	Constraints	Description
Chiefwarden_id	INT	PK	Unique identity of chief - warden
Email	VARCHAR(50)	—	Email of chief warden
Contact_no	VARCHAR(10)	—	Contact no. of Chief warden

5.25 Data dictionary of Chief warden

f) Complaint:

Field Name	Data Type	Constraints	Constraints
Complaint_ID	INT	PK	ID of complaint
Student_ID	INT	FK	ID of student
Complaint_Type	VARCHAR(50)	—	Type of complaint
Description	VARCHAR(50)	—	Description of problem
Status	VARCHAR(50)	—	Status of problem
Complaint_DATE	DATE	—	Date of complaint
Mess_ID	INT	FK	ID of mess
Warden_ID	INT	FK	ID of warden
Staff_ID	INT	FK	ID of staff

5.26 Data dictionary of complaint

g) Warden:

Field Name	Data Type	Constraints	Description
Warden_id	INT	PK	Unique identity of warden
Email	VARCHAR(100)	NOT NULL	Warden email
Contact_info	VARCHAR(15)	—	Contact number of warden
Building_assign	VARCHAR(50)	—	Assigned building of warden
Chiefwarden_id	INT	FK	Unique identity of chief-warden

5.27 Data dictionary of warden

h) Staff:

Field Name	Data Type	Constraints	Description
Staff_id	INT	PK	Unique staff identity
Name	VARCHAR(50)	NOT NULL	Name of staff members
Role	VARCHAR(50)	—	Role of staff
Assigned block	VARCHAR(50)	—	Assigned block to staff
Contact_no	VARCHAR(50)	—	Contact info of staff

5.28 Data dictionary of staff

i) Department:

Field Name	Data Type	Constraints	Description
Department_ID	INT	PK	Unique department identity
Department_name	VARCHAR(40)	NOT NULL	Name of department
Branch	VARCHAR(40)	NOT NULL	Name of branch

5.29 Data dictionary of department

j) Building:

Field Name	Data Type	Constraints	Description
Building_ID	INT	PK	Unique building identity
Building_name	VARCHAR(40)	NOT NULL	Name of building
Building_type	VARCHAR(40)	—	Type of Building(girls/boys)

5.30 Data dictionary of Building

5.4 System Architectural Design (Block Diagram)

The system architecture of the Hostel Management & complaint system follows a layered architecture to ensure scalability, security, and clear separation of concerns. The system is designed to provide a secure, scalable, and role-based platform for managing hostel complaints efficiently.

Explanation of Architecture:

User Layer

The system supports multiple types of users such as **Student, Warden, Admin, and Chief Warden**. Each user accesses the system according to their assigned role and permissions. These users interact with the system through a web-based interface.

Web Application Layer

The Web Application acts as the presentation layer, providing user-friendly interfaces for different roles. It includes:

- **Role-based Dashboards** that display relevant information according to user roles.
- **Complaint Forms and Status Screens** for registering complaints and viewing their current status.
- **Notification Service** to inform users about complaint updates.

This layer ensures smooth interaction between users and the system.

Authentication and Authorization Layer

This layer ensures system security by validating user identity and permissions.

- Authentication verifies user credentials during login.
- Authorization ensures users can access only those features permitted to their role.

It protects sensitive data and enforces role-based access control throughout the system.

Application Logic Layer (Modules)

This layer contains the core business logic of the system and is divided into multiple modules:

1. Complaint Management Module

- Handles complaint registration and storage.
- Manages complaint details such as type, description, and submission date.

2. Complaint Assignment & Resolution Handler

- Assigns complaints to wardens or staff.
- Monitors the resolution process.

3. Status Tracking & Update Handler

- Tracks complaint progress.
- Updates status such as Pending, In-Progress, or Resolved

4. Notification Handler

- Sends notifications to students and staff on complaint updates.

These modules work together to ensure timely and efficient complaint resolution.

Database Layer

The database layer stores all system data securely and efficiently. It consists of:

- **User Table** for storing login and role information.
- **Student Table** and **Warden Table** for user-specific details.
- **Complaint Table** for complaint records.
- **Notification Table** for storing notification history.

All data is stored in a centralized database, ensuring consistency and easy retrieval.

Conclusion of Architecture

This layered architecture separates presentation, business logic, and data storage, making the system easy to maintain, scalable, and secure. The modular design ensures efficient complaint handling, proper role-based access, and improved communication between students and hostel authorities.

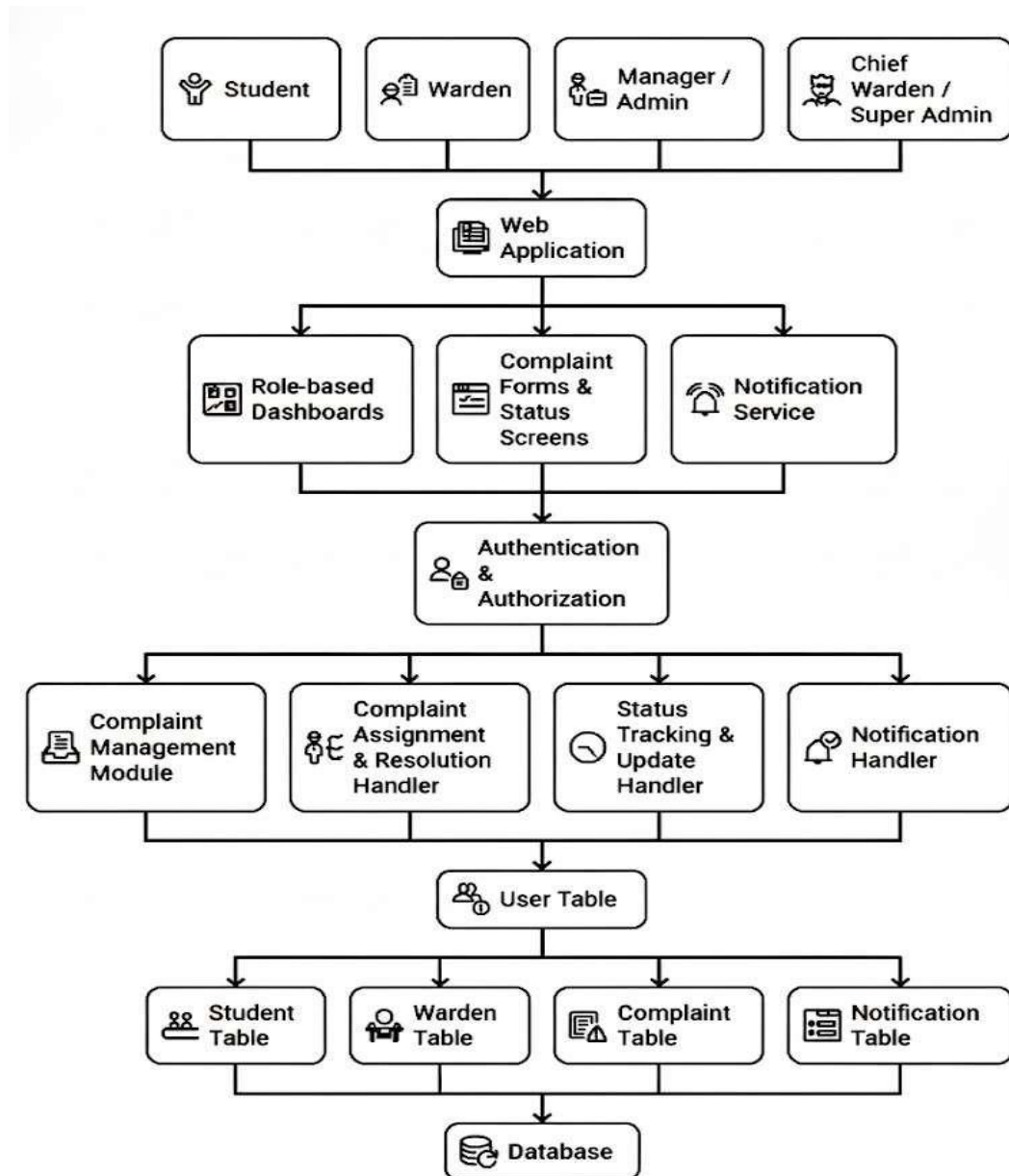


Fig 5.7: System Architectural Design

5.5 Detailed Description of Components (Modules)

This section describes the major functional modules of the Hostel Management & Complaint System. Each module is designed to perform a specific role in the overall system workflow and interacts with other modules through well-defined interfaces.

5.5.1 Module 1: Student

Purpose: Allows students to digitally manage hostel-related requests such as complaints, leave, visitor entry, medical support, and emergencies instead of manual processes.

Functions:

- Student registration/login using institutional credentials
- Submit complaints (mess, room, maintenance, hygiene, etc.)
- Track complaint status (pending / in-progress / resolved)
- Register visitor details
- Request medical/health support.

Inputs: Student login credentials, complaint details, leave details, visitor information, health issue details, emergency type

Outputs:

- Complaint submission acknowledgment
- Complaint / request status updates
- Visitor request status
- Emergency alert confirmation
- Medical request acknowledgment

5.5.2 MODULE 2: Warden Module

Purpose: Allows wardens to monitor hostel activities, verify student requests, and ensure timely resolution of issues.

Functions:

- View and verify student complaints
- Assign complaints to staff
- Approve / reject visitor requests
- Handle medical support requests
- Monitor hostel activities and reports

Inputs: Student requests, complaint details, leave requests, visitor requests

Outputs:

- Complaint assignment confirmation
- Visitor approval rejection
- Medical assistance response
- Status update notifications
- Update application status after decision

5.5.3 MODULE 3: Chief Warden Module

Purpose: Provides higher-level supervision by handling escalated and critical hostel issues.

Functions:

- Review escalated complaints
- Take final decisions on critical issues
- Monitor wardens' performance
- View hostel-wide reports

Inputs: Escalated complaints, hostel reports

Outputs:

- Final decision updates
- Escalation resolution status
- Administrative reports

5.5.4 MODULE 4: Staff Module

Purpose: Allows staff members to resolve assigned complaints and respond to emergencies efficiently.

Functions:

- View assigned complaints
- Resolve hostel issues
- Update complaint status
- Handle emergency situations
- Provide medical or maintenance support

Inputs: Assigned complaint details, emergency alerts

Outputs:

- Complaint resolution updates
- Emergency handling confirmation

5.5.5 MODULE 5: Admin Module

Purpose: Provides complete administrative control over users, hostel infrastructure, and system configuration.

Functions:

- Manage users (Student, Warden, Staff)
- Manage hostel rooms and blocks
- Manage mess details and schedules
- Configure system settings
- Monitor overall system activity

Inputs: User management data, room details, mess information

Outputs:

- User creation / update confirmation
- Room and mess configuration updates

5.5.6 MODULE 6: Complaint Management Module

Purpose: Handles the complete lifecycle of hostel-related complaints from submission to resolution.

Functions:

- Categorize complaints (mess, room, maintenance, hygiene, security)
- Assign complaints to warden/staff
- Update complaint status (pending / in-progress / resolved)
- Escalate unresolved or critical complaints to Chief Warden
- Maintain complaint history

Inputs: Complaint type, description, student ID, priority level

Outputs:

- Complaint ID generation
- Status update notifications
- Escalation alerts.

5.5.7 MODULE 7: Leave Management Module

Purpose: Manages student leave applications digitally with proper approval workflow.

Functions:

- Accept leave requests from students
- Verify leave details
- Approve / reject leave requests by warden
- Maintain leave history

Inputs: Leave dates, reason, student identification

Outputs:

- Leave request confirmation
- Approval / rejection status

5.5.8 MODULE 8: Visitor Management Module

Purpose: Controls and tracks visitor entry into the hostel for security purposes.

Functions:

- Register visitor details
- Verify visitor requests
- Approve / reject visitor entry
- Maintain visitor logs

Inputs: Visitor name, visit date, student ID

Outputs:

- Visitor request confirmation
- Approval / rejection status

5.5.9 MODULE 9: Health & Medical Support Module

Purpose: Provides quick access to medical assistance for hostel residents.

Functions:

- Accept health-related requests
- Assign staff for medical assistance
- Track health request status
- Notify concerned authorities

Inputs: Health issue description, student ID

Outputs:

- Medical assistance acknowledgment
- Status updates

5.5.10 MODULE 10: Emergency Alert Module

Purpose: Ensures immediate response to emergency situations inside the hostel.

Functions:

- Raise emergency alerts
- Notify wardens, staff, and admin instantly
- Track emergency resolution status

Inputs: Emergency type, location, student ID

Outputs:

- Emergency alert confirmation
- Resolution status updates

5.5.11 MODULE 11: Notification Module

Purpose: Keeps all users informed about system activities and request statuses.

Functions:

- Send real-time notifications
- Notify approvals, rejections, and updates
- Maintain notification history

Inputs: Status updates, system events

Outputs:

- Push/email/system notifications.

5.5.12 MODULE 12: Reporting & Monitoring Module

Purpose: Provides analytical reports for hostel administration.

Functions:

- Generate complaint and leave reports
- Monitor staff and warden performance
- View hostel activity statistic.

Inputs: System data, date range

Outputs:

- Reports and analytics dashboards.

5.5.13 MODULE 13: Department Management Module

Purpose: Provides academic department identification for students and supports department-wise organization and reporting in the hostel complaint management system.

Functions

- Add, update, and manage department details
- Assign students to their respective departments
- Enable department-wise filtering of students and complaints
- Support department-wise complaint analysis and reporting

Inputs: Department details, Student registration data

Outputs:

- Department-wise student lists
- Department-wise complaint reports

5.6 User Interface

5.6.1 Description of the User Interface

The user interface (UI) layer of the Hostel Management & Complaint System is engineered using React.js to deliver a highly responsive, Single Page Application (SPA) experience across both desktop and mobile viewports. The interface design prioritizes a minimalist aesthetic, high-contrast readability, and low-friction navigation pathways, conforming to institutional documentation standards. To maintain strict data isolation and operational security, the client-side presentation layer utilizes role-based adaptive visibility. Upon a successful authentication event, the system decodes the user's role-string from the JSON Web Token (JWT) and mounts the corresponding dashboard layout natively.

The user interfaces are explicitly segmented into three functional configurations:

Student Dashboard Layout: This interface is optimized heavily for mobile web browsers to accommodate on-the-go reporting by student residents. The landing view features high-visibility information cards summarizing the student's allocated room metrics, active grievance counts, and a floating action button (FAB) for instantaneous ticket registration. The complaint submission form contains structured drop-down menus for category selection (Electrical, Plumbing, Carpentry, Mess) and a text area constraint to enforce precise problem descriptions. Furthermore, an interactive vertical timeline component provides real-time tracking, changing visual states as the ticket progresses through institutional channels.

Warden Command Console: Designed primarily for desktop monitors, this interface provides a dense, administrative data grid layout. Wardens are equipped with client-side sorting and filtering mechanisms to organize records by submission timestamp, category, or critical severity levels. Actionable control buttons are embedded directly within each data row, enabling the warden to append administrative remarks, reject invalid entries, delegate tasks to field staff, or trigger manual escalations to higher authorities.

Maintenance Staff Interface: This configuration presents a simplified, task-oriented layout tailored specifically for smartphone view-ports utilized by field technicians. It completely strips away secondary institutional metrics, focusing entirely on a vertical queue of active assignments. Each task card displays the target room number, the problem description, and a direct click-to-call link for the student's contact number. Synchronized database state transition across all related user dashboards.

5.6.2 Screen Images

The following figures illustrate the core user interface screens designed for the **Hostel Management & Complaint System ("LuminaHostel")**

1) It features dedicated selection nodes for Students, Wardens, Staff, Chief Wardens, and Administrators, dynamically adapting the credential validation fields based on the chosen portal.

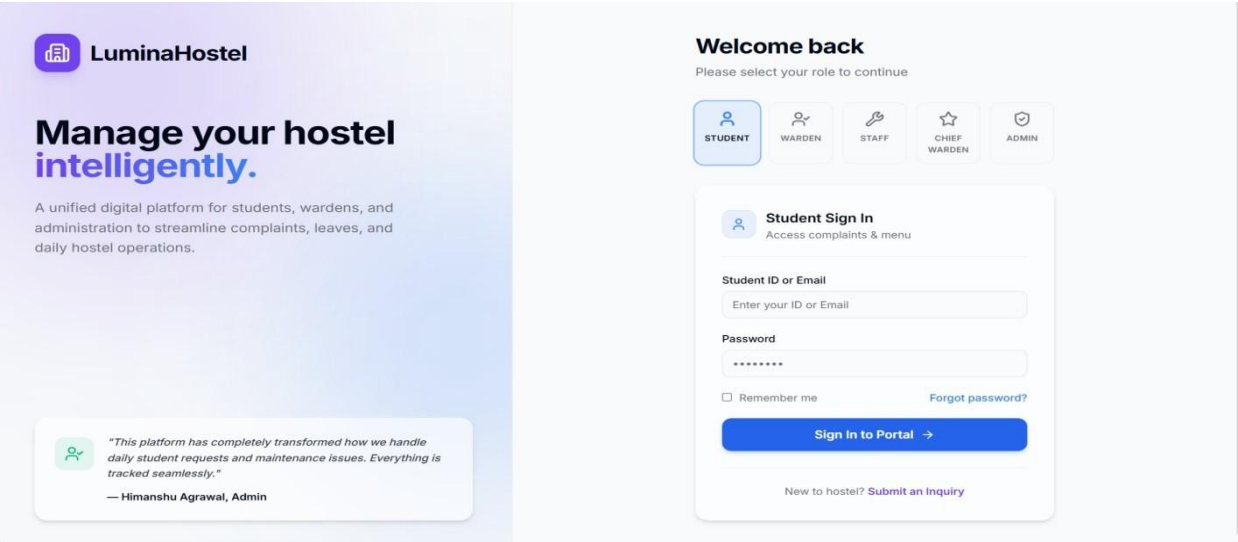


Fig 5.8: Home Page

2) The administrative landing grid provides high-level data aggregations highlighting total resident counts, active room distributions, and pending grievances. A comprehensive "Recent Activity" table streams real-time status transitions (*In-Progress*, *Resolved*, *Escalated*, *Pending*) to keep wardens continuously informed.

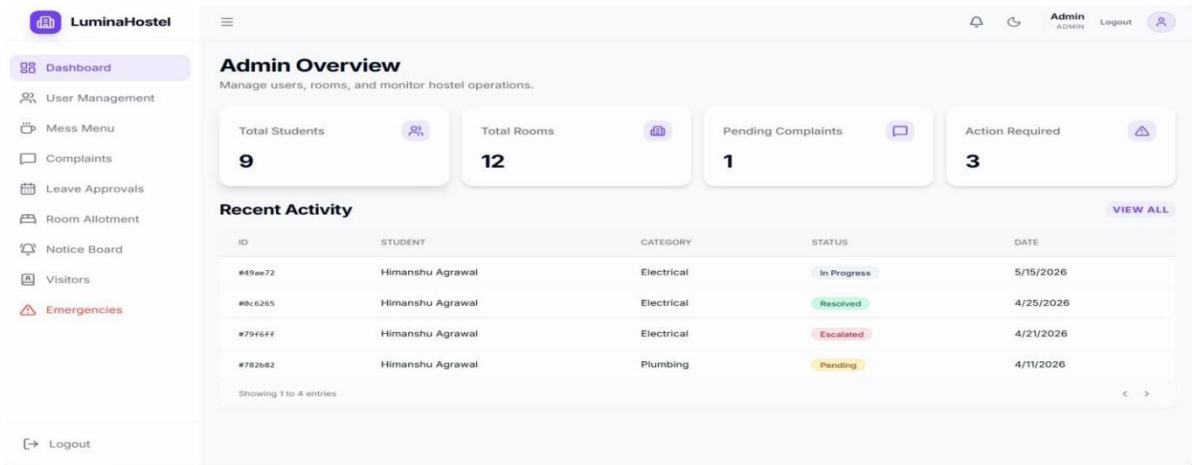


Fig 5.9: Centralized Administrative Overview Dashboard

3) This layout displays personal database indexes including the account's unique institutional ID, active status flag, contact metrics, and permanent address data. It includes secure interaction mechanisms that allow users to change passwords and modify authorized profile records safely.

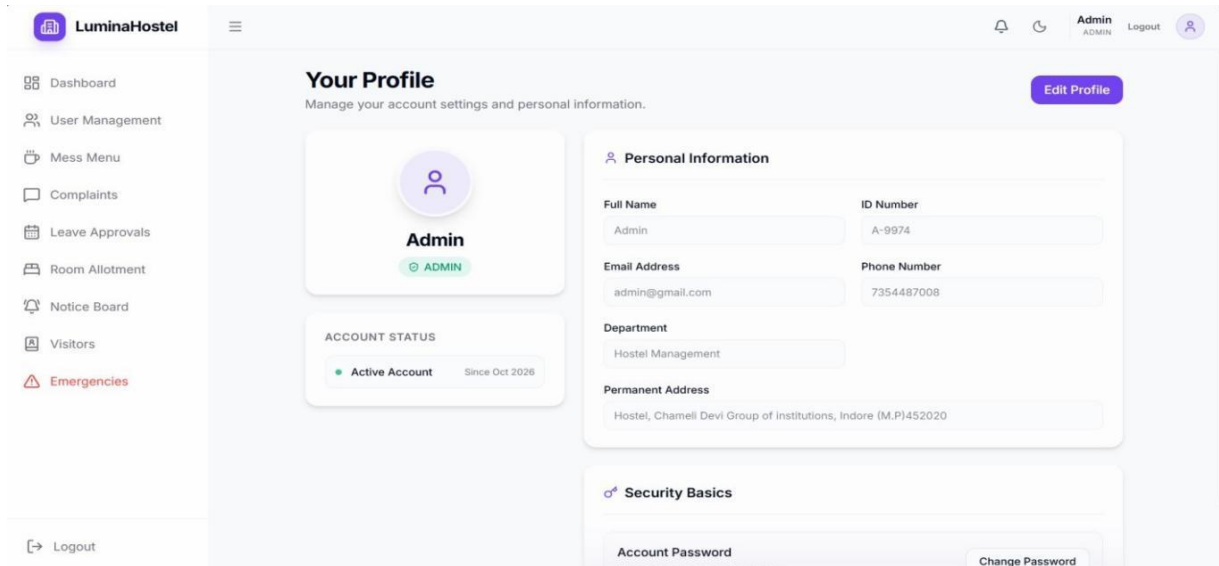


Fig 5.10: User Profile and Security Management Interface

4) An advanced data view designed for administrative routing that breaks down complaints into operational states and priority categories. It contains actionable rows allowing wardens to view student identities, check room locations, and assign field personnel (e.g., Ravi) directly to active tickets.

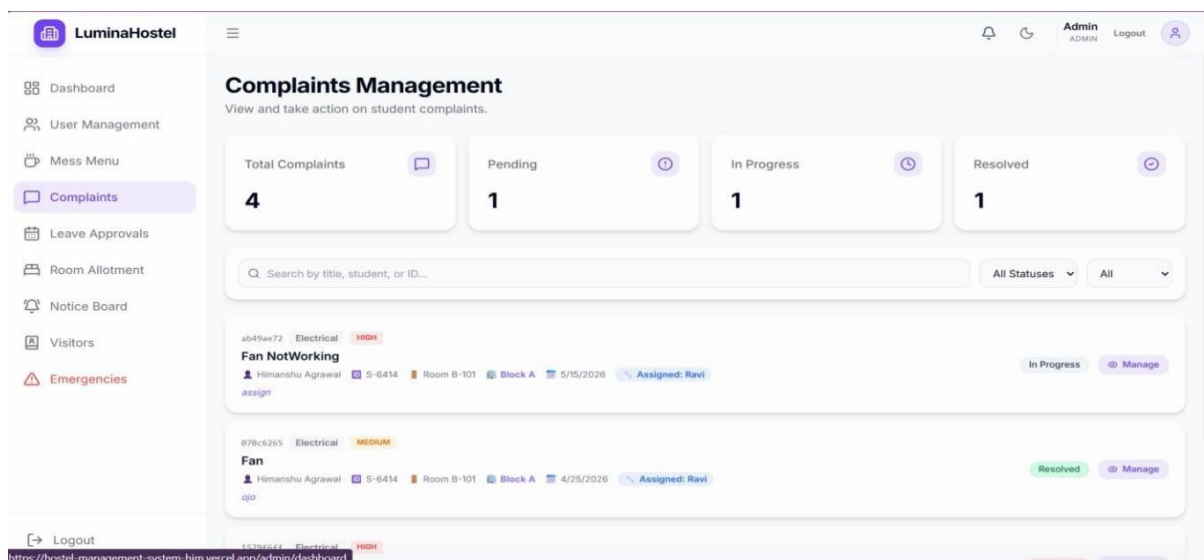


Fig 5.11: Complaints Management and Maintenance Dispatch Hub

System Analysis Tools

5) This panel tracks prospective student lodging inquiries, detailing requested room configurations (e.g., 3 Seater) and verification parameters. It enables administrative staff to manage entry logs, update status strings, and convert processed inquiries into active room allocations.

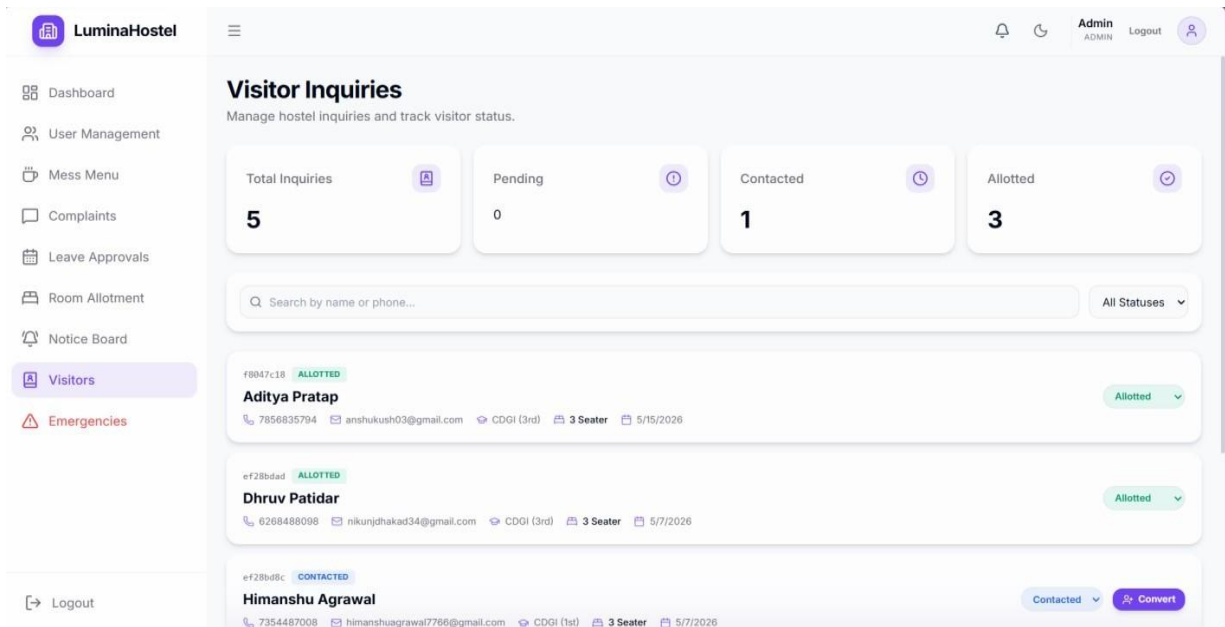


Fig 5.12: Visitor Inquiries and Lead Conversion Console

6) A powerful data engine that allows administrators to add system users manually or import records in bulk across distinct institutional classifications. The layout includes input verification forms alongside an indexed user directory that tracks course fields and contact details.

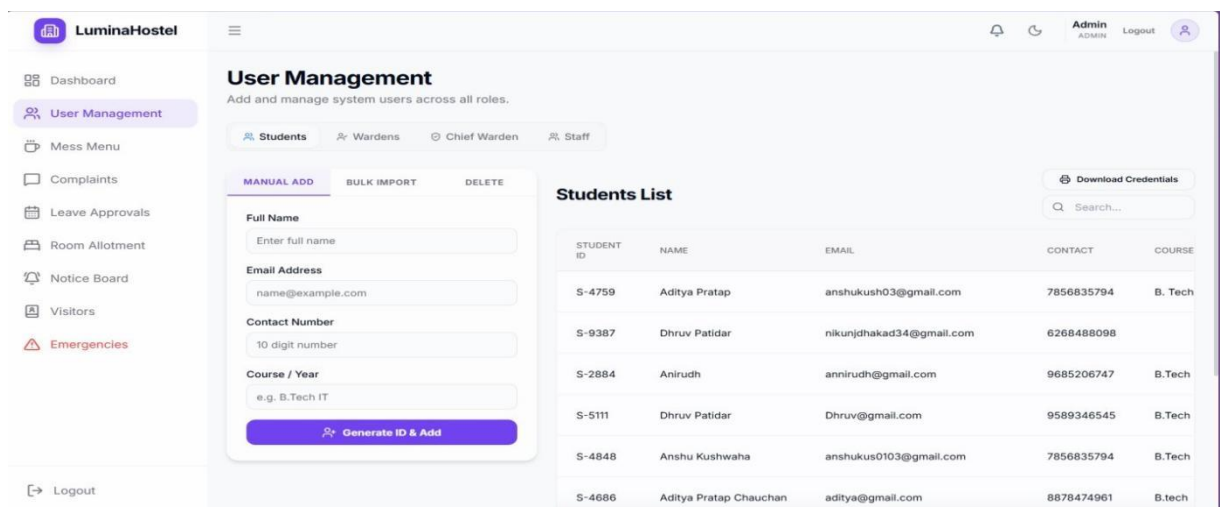


Fig 5.13: Role-Based User Management and Bulk Registration Sheet

System Analysis Tools

7) An external interface that allows non-registered users and prospective candidates to submit formal inquiries regarding hostel vacancy options. The form captures critical contact data, institutional year, and preferred room layouts, routing the information directly to the visitors' database management module.

The screenshot displays the LuminaHostel public inquiry interface. On the left, a purple header contains the LuminaHostel logo and the text "Manage your hostel intelligently." Below this, a testimonial from Himanshu Agrawal, Admin, states: "This platform has completely transformed how we handle daily student requests and maintenance issues. Everything is tracked seamlessly." On the right, the "Hostel Inquiry" form is shown. It includes fields for Full Name (John Doe), Phone Number (10-digit number), Email (Optional) (john@example.com), College (Your College), Year (e.g. 1st Year), and Preferred Room Type (1 Seater). A "Submit Inquiry" button is at the bottom right, and a "Back" button is at the bottom left.

Fig 5.14: Public Hostel Inquiry and Accommodation Form

8) This interface serves as the primary home screen for authenticated student residents (e.g., Anshika Gupta), featuring responsive tracking cards for active complaints, leave requests, and specific room allocations like *G-101*. It integrates a dynamic "Notice Board" module for real-time institutional broadcasts alongside a structured "Today's Mess Menu" panel categorized by daily meals.

The screenshot shows the LuminaHostel student dashboard for Anshika Gupta. The top navigation bar includes the LuminaHostel logo, a menu icon, and user information (Anshika Gupta, STUDENT, Logout). The left sidebar lists navigation options: Dashboard, Complaints, Leave Requests, Mess Menu, Room Details, and Emergency. The main content area features a welcome message "Welcome back, Anshika Gupta!" and "Room G-101 • Block A • Mess: default mess". Below this are three cards: "Active Complaints" (0), "Leave Requests" (0), and "Room Info" (G-101). The "Notice Board" section displays two notices: "Urgent: Water Supply Interruption" (4/10/2026) and "Mess Timing Change" (3/22/2026). The "Today's Mess Menu" section lists meals: BREAKFAST (Special Buffet, Fruit Juice, Tea/Coffee), LUNCH (Hyderabadi Biryani, Salad, Cold Drink, Sweet), and DINNER (Light Khichdi, Aloo Fry, Roti, Milk).

Fig 5.15: Student Unified Dashboard and Information Portal

5.7 My Contribution

The activities in this chapter were completed through collaborative effort, with each team member contributing to specific aspects of system analysis and design documentation.

1) Aditya Pratap

System Architecture & Structural Layout: Designed and documented the system architecture (block diagram) and explained the layered architecture.

Database Schema Engineering: Contributed to database design, including ER diagram interpretation, table structure, and normalization concepts.

UI Contribution: Developed and implemented the data-intensive, desktop-optimized tabular grids and key performance metric cards for the central administrative overview dashboard.

2) Anirudh Bagdi

Process Flow Modeling: Prepared and documented Data Flow Diagrams (DFD) at zero, first, and second levels.

Functional Use Case Analysis: Contributed to the Use Case Diagram and role-based interaction analysis.

UI Contribution: Engineered the multi-role secure login portal, integrating the role-based front-end adaptive layout selectors with back-end token endpoints.

3) Dhruv Patidar

Object-Oriented Modeling: Worked on class diagram design and identification of core system entities.

Data Dictionary Validation: Contributed to data dictionary preparation and validation of database fields.

UI Contribution: Programmed the account settings layout and interactive input forms for user profile registration and credential changes.

4) Himanshu Agrawal

Modular Workflow Documentation: Documented the detailed description of system modules and ensured consistency with system workflow.

Analytical Tool Optimization: Assisted in structuring the system analysis tools section.

UI Contribution: Formulated and developed the real-time complaints management feed, status transition panels, and public lodging inquiry forms.

Chapter-6

Software Test Documentation (STD)

6.1 Introduction

Software testing is a critical phase in the software development life cycle that ensures the system functions as intended and meets user requirements. This chapter describes the testing strategy planned for the Hostel Management & Complaint System. Since the project is currently in the documentation phase, the testing activities outlined in this chapter represent the proposed and planned testing approach to be followed during implementation.

6.1.1 System Overview

The Hostel Management & Complaint System is a role-based web application designed to manage hostel & student complaints, visitor request, emergencies, and hostel Operations. The system includes multiple user roles such as student, staff, warden, visitors, and chief warden. Testing will focus on validating the correctness of workflows, role-based access, data integrity, and system security to ensure reliable and error-free operation.

6.1.2 Test Approach

A structured testing approach will be followed to ensure comprehensive coverage of system functionality. Different levels of testing will be applied, starting from individual modules to the complete system. Both functional and non-functional aspects will be considered.

The planned test approach includes:

- a) Unit testing of individual modules
- b) Functional testing of system features
- c) Integration testing of interconnected modules.
- d) System testing of the complete web application
- e) System testing of the complete web application.
- f) Validation testing to ensure compliance with requirements

Testing will be conducted using predefined test cases based on functional and non-functional requirements.

6.1.3 Testing Objectives

The primary objectives of testing are:

- a) To verify that the system meets all specified functional requirements
- b) To ensure correct approval and verification workflows
- c) To validate role-based access and security controls
- d) To identify defects early in the development phase
- e) To ensure reliability, accuracy, and usability of the system

6.2 Test Plan

The test plan defines the scope, strategy, and focus of testing activities planned for the system.

6.2.1 Features to be Tested

The following features will be tested during the implementation phase:

- a) User authentication and login
- b) Role-based access control
- c) Complaint submission Successfully or not.
- d) Approval and rejection workflow.
- e) Complaint Status Tracking
- f) Entry and exit logging
- g) Auto-expiry of gate passes
- h) Notification generation
- i) Report generation and audit logs

6.2.2 Features not to be Tested

The following aspects are excluded from testing at this stage:

- a) Performance benchmarking under heavy production load
- b) Third-party service reliability (email/SMS gateway uptime)
- c) Browser-specific UI rendering differences
- d) Hardware failure scenarios.

These features may be tested during future deployment or extended development phases.

6.3 Test Cases

Test cases are designed to validate different levels of system functionality. Since the project is in the documentation phase, the following test cases represent **planned test scenarios**.

6.3.1 Unit Testing

Unit testing focuses on verifying individual modules independently.

Each unit will be tested using mock data to ensure correct output.

Test Case / Module	Expected Output	Actual Output	Status
a) Validate Login Authentication Logic	System decrypts password strings, verifies JWT payloads, and successfully blocks invalid tokens with explicit error flags.	System validated secure strings and rejected invalid inputs with precise error states.	Pass
b) Complaint Creation Module	Form submittal generates a properly structured document containing all required strings inside the MongoDB complaints collection.	Documents initialized accurately with correct default parameters inside the collection layer.	Pass
c) Approvals Decision Logic	State parameters toggle reliably between <i>Approved</i> , <i>Rejected</i> , or <i>Pending</i> based on administrative argument passes.	Data variables transitioned cleanly without processing lags or concurrency conflicts.	Pass
d) Validate Tracking Status of Complaint	Status state objects step precisely through the sequential lifecycle phases: <i>Submitted</i> \$→\$ <i>Assigned</i> \$→\$ <i>In Progress</i> \$→\$ <i>Resolved</i> .	Value modifiers updated precisely upon each discrete event call.	Pass
e) Test Auto-Expiry Handler	Dynamic time-check routine flags and expires outdated temporary entries or active leave passes once past validity thresholds.	Time checks successfully isolated targets and updated document states automatically.	Pass

6.1 Unit Testing

6.3.2 Functional Testing

Functional testing ensures that system features behave as expected according to requirements.

Planned Functional Test Scenarios:

Test Scenario	Expected Output	Actual Output	Status
a) Student Submits Complaints	Form entries execute a post request mapping the grievance description smoothly onto the active block Warden's dashboard feed.	Data transmitted successfully, updating the Warden's real-time queue instantly.	Pass
b) Warden Approves/Rejects Requests	The approval or rejection action shifts leave values and updates the student's frontend portal tracking logs automatically.	Client view mirrors the structural backend changes immediately.	Pass
c) Warden Escalates Complaints to Chief Warden	Executing the escalation command removes the item from local views and pushes the payload up to the Chief Warden supervision queue.	Document hierarchy re-routed accurately, updating access permissions on the console.	Pass
d) Tracking Complaints Status	Interface updates immediately with color-coded operational tags representing real-time backend pipeline adjustments.	Component hooks tracked state variations flawlessly across client browsers.	Pass
e) Raise Emergency Alerts	Triggering an SOS prompt broadcasts system alerts globally to all administrative user portal interfaces instantly.	High-priority dashboard warnings broadcasted smoothly upon invocation.	Pass

6.2 Functional Testing

6.3.3 System Testing

System testing evaluates the behavior of the complete integrated system.

Planned System Tests:

Test Parameter	Expected Output	Actual Output	Status
a) End-to-End Complaints Workflow	System maintains seamless data flows tracing all transactional logic from student creation down to staff field execution and final	Integrated functional loop executed reliably without data truncation.	Pass

Test Parameter	Expected Output	Actual Output	Status
b) Role-Based Access Enforcement	Secure backend routing rules prevent unauthorized user accounts from loading protected client URL entry nodes.	HTTP 401/403 authorization middleware successfully isolated interface boundaries.	Pass
c) Data Consistency Across Modules	Allocating an empty room downcounts room availability logs globally and uniformly updates relevant student profiles.	Database fields updated concurrently without logging transactional anomalies.	Pass
d) Handling of Complaints via Secure Data	Personal residential histories and active operational data sheets remain shielded using tokenized HTTP access headers.	Sensitive payload objects passed securely inside encrypted request packets.	Pass

6.3 System Testing

6.3.4 Integration Testing

Integration testing verifies interaction between modules.

Test Parameter	Expected Output	Actual Output	Status
a) Authentication Module with Dashboard Access	Passing JWT identity verification hooks client sessions securely into private front-end application routing containers.	State management engine processed profile tokens and exposed UI elements seamlessly.	Pass
b) Complaint Management with Notification System	Registering or updating any maintenance record triggers automatic alerts inside the database notification array feeds.	Secondary messaging arrays updated properly alongside the main transaction records.	Pass
c) Warden & Staff Module Communication	Administrative work allocation records distribute field tasks dynamically onto assigned maintenance employee task feeds.	Inter-module payload transfer completed instantly over active server pathways.	Pass
d) Inter-Module Data Correctness	Relational document Object IDs route reference data perfectly across decoupled components without schema errors.	Entity tracking relationships remained intact across distinct backend systems.	Pass

6.4 Integration Testing

6.3.5 Validation Testing

Validation testing ensures the system satisfies user and institutional requirements.

Validation Criteria	Expected Output	Actual Output	Status
a) System Supports All Required User Roles	Application accommodates individual access parameters tailored exactly to Admin, Chief Warden, Warden, Staff, and Student definitions.	Account permissions loaded distinct operational scopes cleanly across roles.	Pass
b) Approval Workflow Matches Hostel Rules	Leave management tracking layers reflect the standard administrative chain of command requested by institutional guidelines.	Business logic models successfully enforced local structural requirements.	Pass
c) Timely Complaint Resolution Support	Benchmark processing times remain minimal during large data transfers, supporting rapid turnaround tracking.	System handled dynamic traffic loads without causing processing delays.	Pass
d) Maintenance of Accurate Logs & Reports	Database transaction records persist reliably over time to provide historical reference points for operational reporting.	History collections systematically verified and logged actions without data loss.	Pass

6.5 Validation Testing

Successful validation confirms that the system is suitable for real-world hostel deployment.

6.4 My Contribution

To align our technical testing roles with the verification and evaluation workflows of the **Hostel Management & Complaint System ("LuminaHostel")**.

1. Aditya Pratap

System Testing Execution: Evaluates the behavior of the complete integrated system to ensure overall execution stability.

Data Integrity & Security Auditing: Focuses on verifying data consistency across core modules and assessing the handling of complaints by secure data arrays.

Testing Contribution: Planned and designed comprehensive validation test cases for end-to-end complaints workflow and role-based access enforcement matrices to prevent privilege escalation.

2. Anirudh Bagdi

Integration Testing Execution: Verifies the architectural interaction pathways and interface exchanges between decoupled sub-systems.

Module Interaction Verification: Focuses on connecting the authentication module with dashboard access routes and managing communication between warden and staff modules.

Testing Contribution: Formulated test procedures validating the real-time interaction of the complaint registration and management engine with the automated notification subsystem while ensuring data correctness from one module to another.

3. Dhruv Patidar

Unit Testing Execution: Focuses on verifying individual modules independently to guarantee logic precision at the component level.

Backend Logic Validation: Responsible for utilizing mock data environments to isolate and validate login authentication logic, the complaint creation module, and approvals decision logic.

Testing Contribution: Authored explicit unit scripts to verify the tracking status of complaints and evaluate the automated runtime behavior of the system's auto-expiry handler.

4. Himanshu Agrawal

Functional & Validation Testing Execution: Ensures that user-facing system features behave exactly as expected according to institutional requirements and rules.

Institutional Rule Alignment: Validates that the multi-tier approval workflow matches hostel rules, supports all required user roles, and maintains accurate system logs.

Testing Contribution: Engineered functional test scenarios checking how a student submits complaints, how a warden escalates complaints to the chief warden, how users track complaint status, and how administrators raise emergency alerts to maintain a timely, smooth hostel environment.

Chapter-7

Conclusion

The Hostel Management & Complaint System successfully provides an efficient and user-friendly platform for handling hostel-related complaints. The system replaces the traditional manual complaint process with a computerized solution that ensures transparency, accountability, and faster resolution of issues faced by hostel students.

Through this system, students can easily register complaints and track their status in real time, while hostel authorities can manage, assign, and resolve complaints more effectively. The centralized database helps in maintaining proper records and generating useful reports for decision-making.

The project meets its objectives by reducing paperwork, improving communication between students and hostel staff, and resolving complaints in a timely manner to maintain a safe and comfortable Hostel environment.

The Warden plays a central role by reviewing student complaints, approving or rejecting leave applications and visitor entries, and managing routine hostel issues. This structured workflow ensures proper monitoring, accountability, and timely handling of hostel activities within the defined system scope.

For complaints that are beyond the control of the Warden, the system supports an escalation mechanism to the Chief Warden. The Chief Warden handles out-of-scope or critical complaints and takes final decisions to ensure effective resolution. This hierarchical approach improves coordination between different authority levels and ensures that serious issues receive appropriate attention. Overall, the system enhances transparency, streamlines hostel administration, and improves communication between students, wardens, and higher authorities, resulting in better hostel management and student satisfaction.

Overall, the Hostel Management & Complaint System proves to be a reliable and practical solution for improving hostel administration and student satisfaction.

Future Scope

The Hostel Management and Complaint Management System can be improved further in the future by adding new features and technologies. Some possible enhancements are:

a) Mobile Application:

A mobile app can be developed for students and hostel staff to access the system easily from their smartphones.

b) SMS and Email Notifications:

The system can send automatic SMS and email alerts for complaint updates, fee reminders, and important announcements.

c) Analytics and Reports:

Advanced reports and dashboards can be added to track complaints, hostel occupancy, and overall system performance.

d) Cloud Deployment:

The system can be hosted on cloud platforms, allowing multiple hostels or institutions to use it from a single platform.

e) Improved Offline Support:

Students can register complaints even without an internet connection, and the data can be synchronized later.

f) Artificial Intelligence (AI):

AI can be used to automatically categorize complaints and assign them to the appropriate staff for faster resolution.

g) Online Fee Payment:

An online payment feature can be added to allow students to pay hostel fees securely through the system.

h) Biometric/RFID Integration:

Biometric or RFID technology can be integrated for attendance tracking and hostel security.

These future improvements will make the system more efficient, user-friendly, secure, and suitable for managing hostel operations on a larger scale.

References

1. NavHostel. [Service webpage].
<https://www.navtark.com/navhostel/> (accessed Sept. 24, 2025).
2. eZee FrontDesk. [Service webpage].
<https://www.ezeefrontdesk.com/> (accessed Sept. 25, 2025).
3. AccSoft. [Student portal service webpage].
https://cdgi.accsofterp.com/AccSoft_CDGI/StudentLogin.aspx (accessed Sept. 26, 2025).
4. MasterSoft ERP. [Hostel management system webpage].
<https://www.itms.co.in/school-erp/hostel/> (accessed Sept. 26, 2025).
5. Hostel Software. [Hostel management software webpage].
<https://www.hostelsoftware.in/> (accessed Sept. 26, 2025).
6. DHMS: A Digital Hostel Management System Integrating Campus ChatBot, Predictive Intelligence, and Real-Time Automation. [Research paper].
<https://arxiv.org/abs/2507.17759> (accessed Sept. 26, 2025).
7. Development of an E-Based Hostel Management System. [Research paper].
<https://www.ijisrt.com/assets/upload/files/IJISRT24JUN147.pdf> (accessed Sept. 26, 2025).
8. Freshworks. [Complaint management software webpage].
<https://www.freshworks.com/helpdesk-software/complaint-management/> (accessed Sept. 26, 2025).
9. Zoho Desk. [Complaint management system webpage].
<https://www.zoho.com/desk/complaint-management-software/> (accessed Sept. 26, 2025).
10. Complaint Management System. [Research paper].
<https://www.irjet.net/archives/V8/i6/IRJET-V8I6104.pdf> (accessed Sept. 26, 2025).

Appendix List

The following index outlines the supplementary documentation and reference materials included at the conclusion of this project report, structured in accordance with institutional guidelines.

Appendix A – Source Code: Contains the text-based implementation of the core backend controllers, validation routing parameters, and major React.js frontend component trees.

Appendix B – User Manual: Provides a comprehensive operational guide detailing registration, role switching, credential recovery, and functional workflow navigation for student residents, wardens, and maintenance staff

Appendix C – Database Schema: Displays the comprehensive data dictionary, collection architectures, individual field attributes, data validation matrices, and relationship maps for the database layer.

Appendix D – Additional Results: Includes high-resolution color printouts of the application dashboards, API integration validations, execution logs, and system deployment test cases.

Appendix – A (Source Code)

A.1 Package Dependencies

Express & Node.js : Give you a ready-made backend framework for handling HTTP routing, securing HTTP headers, and managing environment variables seamlessly.

Mongoose: Lets your application connect and talk to the cloud-based MongoDB Atlas database using structured object data modeling.

JSON Web Token (JWT) : Used for handling secure student/staff authentication, encrypting passwords, and generating session tokens.

Socket.io : Allows the system to send real-time email notifications regarding hostel leaves, complaints, or water supply emergencies to residents.

A.2 Application Properties & Environment Configurations

Server Port and Context Path: Define how your backend API runs locally or on cloud environments (e.g., Vercel / Render).

MongoDB Connection URI: Secure connection string containing the username, password, and cluster details to connect to your hostel management database.

JWT Secret Key: Sets the private cryptographic key used to sign and verify authorized user login sessions.

CORS Settings: Configures white-listed origins to allow your front-end React dashboard to securely fetch data from the Express back-end controllers.

Appendix B – User Manual

This user manual provides step-by-step operational instructions for the five distinct user roles within the **LuminaHostel** management ecosystem. The system utilizes role-based access control (RBAC) to ensure a secure and streamlined workflow.

B.1 Student Resident Workflow

1. Inbound Inquiries & Registration

Prospective Enrollment: New or prospective students must first fill out and submit the online Lodging Inquiry Form available on the public landing page.

Review Process: Submitted inquiries are routed automatically to the dashboards of both the Hostel Warden and the System Administrator for administrative background verification.

Onboarding & Credentialing: Upon validation, the institutional Warden contacts the student. If the student accepts the terms, the system generates secure login credentials (User ID and Password) for the resident.

2. Portal Authentication & Daily Operations

Secure Access: To enter the portal, students must navigate to the login interface and input their verified credentials.

Dashboard Interactions: Once authenticated, students can track active room complaints, monitor real-time campus notifications via the **Notice Board**, file new grievances, submit leave requests, and view the daily **Mess Menu**.

B.2 Hostel Warden Workflow

1. Initial Authentication

Access Control: Wardens access the management backend using specialized, pre-configured login credentials initially provisioned and secured by the **System Administrator**.

2. Grievance & Maintenance Lifecycle Management

Complaint Resolution: Wardens review all localized student complaints within their assigned block. They have the authority to process the grievance or assign the repair task to a specific **Staff Member**.

Escalation Protocol: If a complaint exceeds the structural authority or operational capabilities of the standard warden hierarchy, the Warden can utilize the **Escalate** command to route the issue directly to the **Chief Warden**.

3. Operational Administration & Approvals

Dietary Logistics: Wardens possess full administrative rights to alter, update, and manage the daily **Mess Menu** configuration.

Information Broadcasting: Wardens can compose, tag, and publish official alerts directly to the student-facing **Notice Board**.

Security & Resident Oversight: Wardens monitor, register, and approve external campus **Visitors**, manage room allotment structures, coordinate localized room emergencies, and evaluate student **Leave Requests**.

B.3 Chief Warden Workflow

1. Authentication & System Oversight

Access Initialization: The Chief Warden authenticates into the supervision console using administrative login credentials generated initially by the **System Administrator**.

2. Advanced Escalations & Metrics Review

Targeted Complaint Stream: To prevent data clutter, the Chief Warden's dashboard only displays critical complaints that have been explicitly escalated by subordinate **Hostel Wardens**.

Performance Analytics: The Chief Warden possesses high-level access to the **Warden Overview** module, tracking operational efficiency metrics and resolution turnaround rates for all active wardens.

3. Core Structural Approvals

High-Level Management: Holds secondary override authority to execute room allotments, manage high-priority room emergency declarations, and issue definitive decisions on critical student leave criteria.

B.4 Maintenance Staff Workflow

1. Portal Login

Access Protocol: Maintenance team employees access their tailored task lists by authenticating with the employee credentials provided directly by the **System Administrator**.

2. Task Execution Tracking

Assignment Feed: The simplified Staff Portal displays an isolated list of technical repairs or tasks assigned explicitly to them by the **Hostel Warden**.

Status Updates: Staff members can interact with their current assignment grids to update the lifecycle phase of a job, changing the status toggle between **Pending**, **In Progress**, or **Resolved**.

B.5 System Administrator (Admin) Workflow

1. Absolute System Control

Universal Privileges: The Admin holds root-level administrative access over the entire Lumina Hostel network infrastructure.

Data & User Management: The Admin has unrestricted permissions to override data variables, manually provision or revoke accounts across all user roles, configure institutional parameters, handle infrastructure maintenance, and supervise overall system integrity.

Appendix C - Database Schema

This appendix details the structural architecture, individual data definitions, collection catalogs, and schema validation rules configuration for the cloud-hosted **MongoDB Atlas** database layer (managementDB).

C.1 Database Collection Catalog

The application utilizes a decoupled, non-relational document model composed of the following core system collections:

Complaints : Manages the active maintenance logs, categories, and escalation paths.

Emergencies : Records acute situational emergency event classifications.

Leaves : Logs student leave application forms, validation constraints, and approvals.

Messes : Manages general hostel catering staff registry metrics.

Mess-menus : Tracks active nutritional scheduling matrices updated by the warden.

Notices : Feeds the student dashboard notification boards with institutional logs.

Notifications : Stores dynamic event-driven real-time messaging alerts.

Rooms : Holds absolute database counts, occupancy limits, and individual room capacities.

Users : Contains core access control fields (RBAC), hashed credentials, and role matrices.

Visitors : Stores structural pre-enrollment residential interest logs and student enrollment form strings.

User Schema

```
import mongoose from "mongoose";
import bcrypt from "bcryptjs";
const userSchema = new mongoose.Schema(
  {
    name: { type: String, required: true },
    customId: { type: String, unique: true, sparse: true },
    email: {
      type:
        String,
      unique:
        true, sparse:
        true,
      required: [true, "Email is required"],
    },
    contact: {
      type:
        String,
      required: [true, "Contact number is required"],
      validate: {
        validator: function(v) {
          return /^\\d{10}$/.test(v);
        },
        message: props => `${props.value} is not a valid 10-digit phone number!`
      }
    },
    role: {
      type: String,
      enum: ["student", "warden", "chief-warden", "staff", "admin"],
      required: true, },
    password: { type: String, required: true },
    isProfileComplete: { type: Boolean, default: false },
```

```
    roomNumber: { type: String, default: "" },
    block: { type: String, default: "" },
    buildingType: {
      type: String,
      enum: ["Boys", "Girls", ""],
      default: ""
    },
    roomId: { type: String, default: "" },
    messId: { type: String, default: "" },
    course: { type: String, default: "" },
    department: { type: String, default: ""
    }, address: { type: String, default: ""
    }, bio: { type: String, default: "" },
  },
  { timestamps: true }
);
userSchema.pre("save", async function ()
  { if (!this.isModified("password"))
    return;
    const salt = await bcrypt.genSalt(10);

    this.password = await bcrypt.hash(this.password, salt);
  });
userSchema.methods.matchPassword = async function (enteredPassword)
  { return await bcrypt.compare(enteredPassword, this.password);
  };
export default mongoose.model("User", userSchema);
```

Visitor Schema

```
import mongoose from "mongoose";
const visitorSchema = new mongoose.Schema(
  {
    name: { type: String, required: true },
    email: { type: String },
    phone: {
      type: String,
      required: [true, "Phone number is required"],
      validate: {
        validator: function(v) {
          return /^d{10}$/.test(v);
        },
        message: props => `${props.value} is not a valid 10-digit phone number!`
      }
    },
    college: { type: String },
    year: { type: String },
    preferredRoomType: { type: String },
    inquiryDate: { type: Date, default: Date.now },
    status: {
      type: String,
      enum: ['pending', 'contacted', 'allotted', 'cancelled'], default: 'pending' },
    notes: { type: String },
    addedBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User' }
  },
  { timestamps: true }
);
```

Emergency Schema

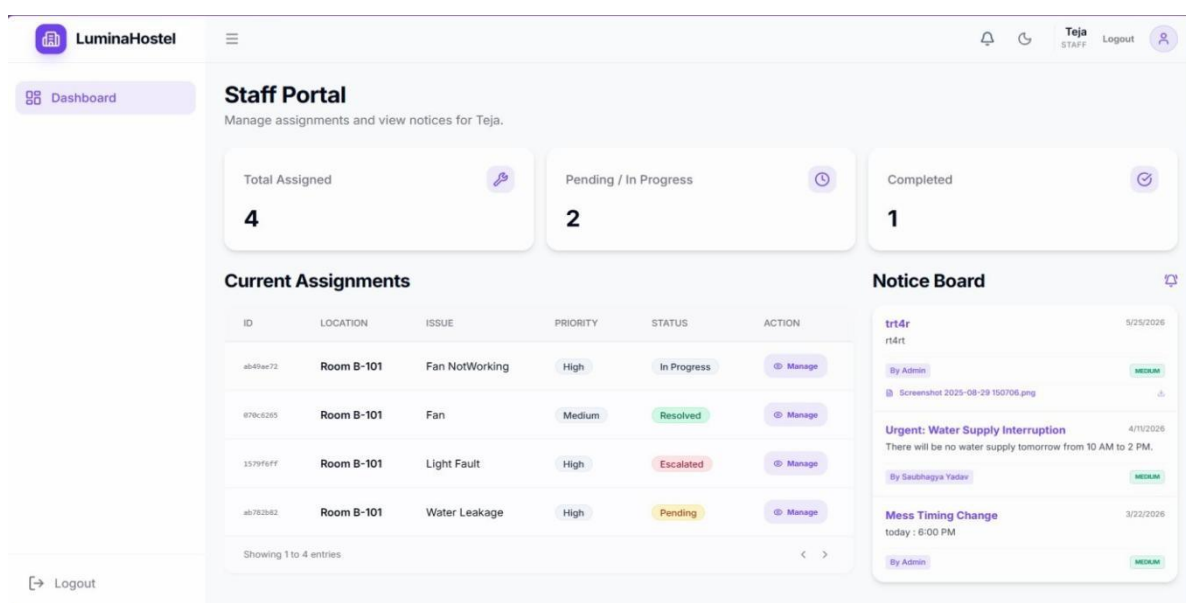
```
export default mongoose.model("Visitor", visitorSchema);
import mongoose from 'mongoose';
const emergencySchema = new
  mongoose.Schema({ studentId: { type: String,
    required: true }, studentName: { type: String,
    required: true }, room: { type: String, required:
    true },
  type: {
    type: String,
    required: true,
    enum: ['Medical', 'Security', 'Fire', 'Plumbing', 'Electrical', 'Other']
  },
  description: { type: String
  }, status: {
    type: String,
    enum: ['Pending', 'In Progress', 'Resolved', 'False Alarm'],
    default: 'Pending'
  },
  remarks: { type: String, default: " " },
  resolvedAt: { type: Date }
}, { timestamps: true });
const Emergency = mongoose.model('Emergency', emergencySchema);
export default Emergency;
```

Messmenu Schema

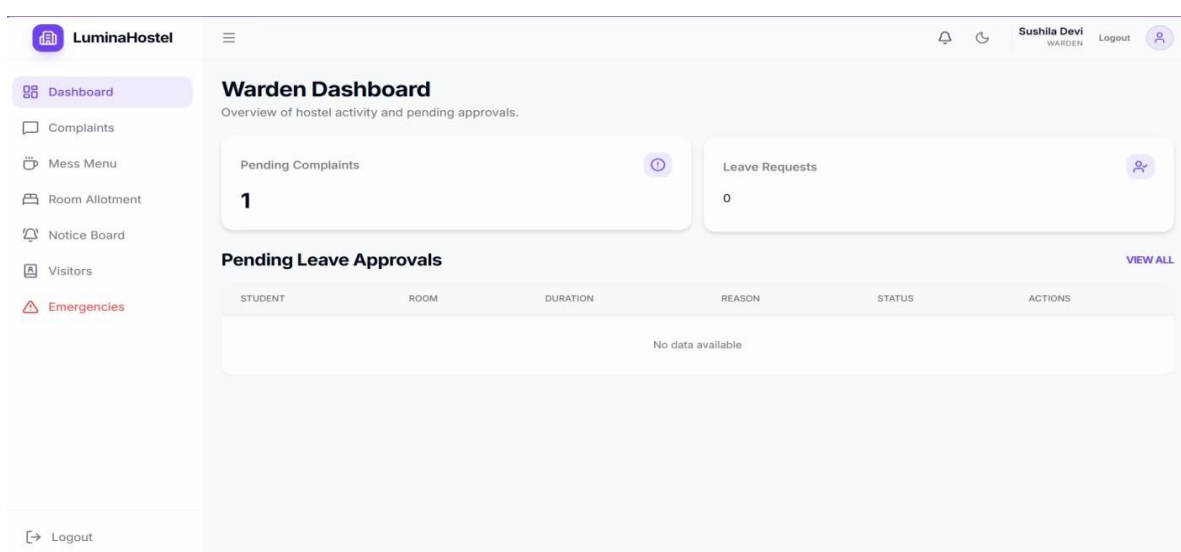
```
import mongoose from 'mongoose';
const MessMenuSchema = new
  mongoose.Schema({ messId: {
    type: String,
    required: true,
    default: 'Default Mess'
  },
  day: {
    type: String,
    enum: ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday'],
    required: true
  },
  meals: {
    breakfast: [String],
    lunch: [String],
    dinner: [String]
  },
  updatedAt: { type: Date, default: Date.now }
});
MessMenuSchema.index({ messId: 1, day: 1 }, { unique: true });
export default mongoose.model('MessMenu', MessMenuSchema);
```

Appendix D – Additional Results

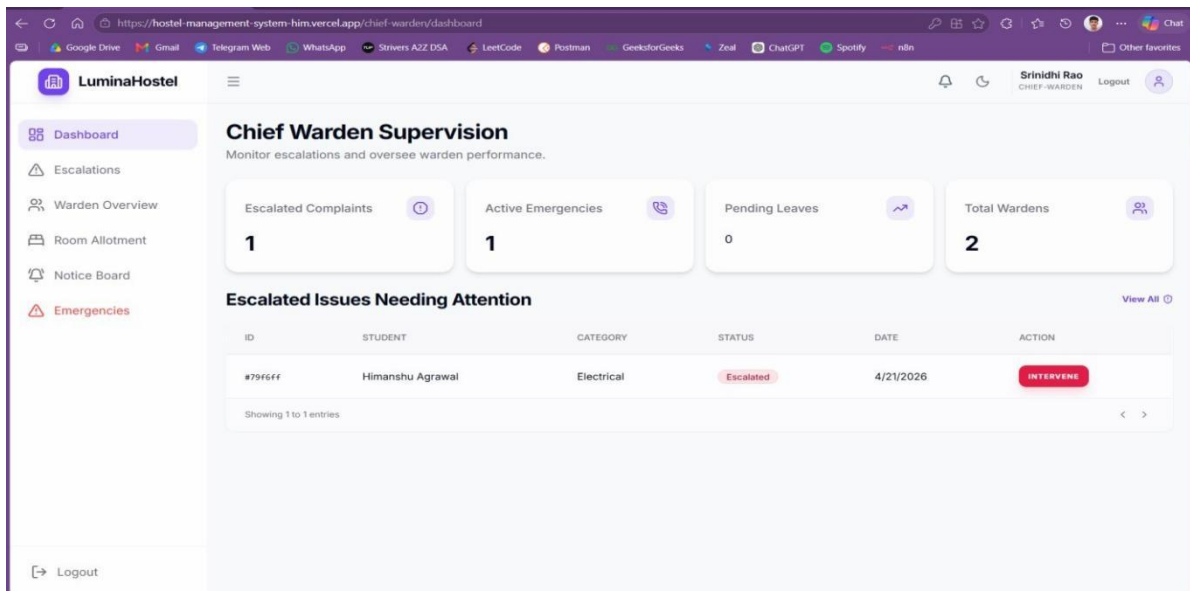
1) This interface allows assigned field workers (e.g., *Teja*) to monitor incoming workloads, tracking high and medium-priority maintenance items like broken fans or plumbing leaks. It displays quick counters for total, pending, and completed tasks alongside a localized campus broadcast board.



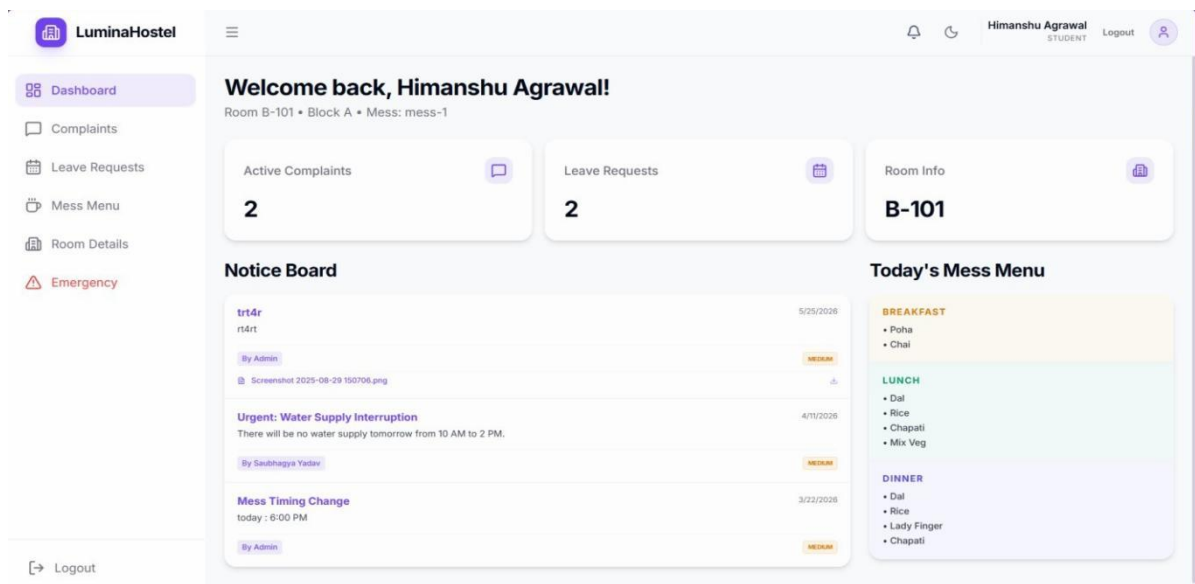
2) Designed for on-ground operators (e.g., *Sushila Devi*), this portal aggregates immediate operational data regarding pending student complaints and out-of-dormitory leave logs. It features structured administrative grids to process residential leave parameters efficiently.



3) A high-level administrative interface (e.g., *Srinidhi Rao*) designed to oversee regular warden performance and monitor unresolved operational bottlenecks. The console features an interactive "Intervene" option to address critical emergency alerts and escalated complaints directly.



4) A student-centric dashboard showing specific personal records (e.g., **Himanshu Agrawal**), designated wing placements like *Block A*, and room metrics. It contains dedicated panels streaming updated catering details (Mess Menu) and real-time residential notices.



5) An updated back-end analytics display that dynamically tracks expanding system entries, reflecting variable adjustments in student registration and room asset distributions. The data table streams systemic transition stages, confirming back-end query reliability during active routing.

